



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس شبکه‌های عصبی و یادگیری عمیق
پاسخ تمرین شماره چهار

دانشجویان:

سوال یک: احمد رضا نوپوش شماره دانشجویی ۶۱۰۳۰۱۱۹۴

سوال دو: رضا مردانی شماره دانشجویی: ۶۱۰۳۰۱۱۸۰

فهرست مطالب

۴	فهرست تصاویر
۵	فهرست جداول
۶	۱ مدل‌های RNN برای وظایف مشترک درک زبان گفتاری
۶	۱-۱ آماده‌سازی داده
۶	۱-۱-۱ <PAD> و <UNK>
۸	۲-۱ BiRNN
۸	۱-۲-۱ معماری مدل BiRNN
۸	۲-۲-۱ نتایج آموزش
۹	۳-۲-۱ گزارش دسته‌بندی Intent
۹	۴-۲-۱ گزارش دسته‌بندی Slot
۱۳	۵-۲-۱ تفسیر نتایج
۱۳	۳-۱ BiLSTM
۱۴	۱-۳-۱ نتایج روی دیتای تست
۱۵	۲-۳-۱ گزارش دسته‌بندی هدف (Intent Classification)
۱۵	۳-۳-۱ Slot Classification
۱۸	۴-۳-۱ تفسیر نتایج
۱۹	۴-۱ Decoder Encoder
۱۹	۱-۴-۱ تفاوت بنیادین معماری BiLSTM و Encoder-Decoder
۲۰	۲-۴-۱ نقش Decoder و اثر افزایش عمق آن در مدل
۲۰	۳-۴-۱ ضرورت استفاده از توکن‌های <BOS> و <EOS>
۲۱	۴-۴-۱ نتایج آموزش و ارزیابی عملکرد مدل روی داده‌ی آزمون
۲۶	۵-۴-۱ تحلیل علت ضعف مدل در تشخیص Slot
۲۶	۶-۴-۱ روش پیشنهادی مقاله برای رفع این مشکل

۲۷	۵-۱ تحلیل مقایسه‌ای عملکرد مدل‌ها در Slot Filling
۲۷	۱-۵-۱ تحلیل دقیق خطاهای Seq ² Seq با مثال‌های واقعی Greedy Decoding
۳۴	۲ پیش‌بینی قیمت سهام با استفاده از مدل‌های Recursive
۳۴	۱-۲ پیش‌پردازش داده‌ها
۳۴	۱-۱-۲
۳۷	۲-۱-۲
۴۱	۳-۱-۲
۴۲	۴-۱-۲
۴۴	۲-۲ مدل پایه
۴۴	۱-۲-۲ نمودارها
۵۰	۲-۲-۲
۵۲	۳-۲ مدل‌های Recursive Neural Network
۵۲	۱-۳-۲ معماری LSTM
۵۷	۲-۳-۲ معماری GRU
۶۱	۴-۲ مقایسه روی داده‌های تست و مقایسه عملکرد
۶۱	۱-۴-۲
۶۱	۲-۴-۲
۶۲	۳-۴-۲
۶۴	۵-۲ بخش امتیازی
۶۴	۱-۵-۲
۶۶	۲-۵-۲

فهرست تصاویر

۸		۱	نمودار آموزش BiRNN
۱۴		۲	نمودار آموزش BiLSTM
۲۱		۳	نمودار آموزش Encoder-Decoder

فهرست جداول

۹	نتایج کلی BiRNN روی داده‌های تست	۱
۹	BiRNN for Report Classification Intent	۲
۱۰	BiRNN for Report Classification Slot	۳
۱۴	نتایج مدل BiLSTM روی دیتای تست	۴
۱۵	BiLSTM for Report Classification Intent	۵
۱۵	BiLSTM for Report Classification Slot	۶
۲۱	نتایج آزمایش مدل SEQ ^۲ SEQ	۷
۲۲	Report Classification Intent	۸
۲۲	گزارش طبقه‌بندی اسلات‌ها (Slot Classification Report)	۹
۲۷	مقایسه‌ی کیفی رفتار سه مدل در Filling Slot	۱۰
۶۱	جدول مقایسه عملکرد مدل‌ها روی داده‌های تست	۱۲
۶۱	مقایسه کارایی محاسباتی مدل‌ها	۱۳
۶۵	ساختار لایه‌های شبکه CNN-Only	۱۴
۷۰	مشخصات معماری، پنجره‌های زمانی و ویژگی‌های ورودی مدل CNN-GRU	۱۱

۱ مدل‌های RNN برای وظایف مشترک درک زبان گفتاری

۱-۱ آماده‌سازی داده

برای انجام این کار، از نسخه‌ای تمیز و ساخت‌یافته از مجموعه‌داده ATIS (سیستم اطلاعات سفرهای هوایی) استفاده شده است. در این سوال ما با دو مأموریت مواجه هستیم: تشخیص نیت (Intent Detection) که به معنای طبقه‌بندی جمله مسافر به یکی از ۱۷ کلاس موجود است و مأموریت بعدی ما تشخیص اسلات‌های جمله مسافر که به این معنی است هر کلمه جمله مسافر دارای چه معنایی است؟

مجموعه‌داده شامل جملات طبیعی کاربران مرتبط با سفرهای هوایی است، که هر جمله دارای یک برچسب نیت و یک توالی برچسب اسلات مطابق با روش BIO می‌باشد. قالب BIO به ما امکان می‌دهد تا اجزای معنایی جمله را به‌صورت دقیق شناسایی کنیم: برچسب B برای شروع یک اسلات، I برای ادامه همان اسلات، و O برای کلمات غیرمرتبط با هیچ اسلاتی استفاده می‌شود. این روش تضمین می‌کند که مدل بتواند به‌طور دقیق و توکن به توکن اطلاعات معنایی مهم را استخراج کند.

پس از بارگذاری مجموعه‌داده، ابتدا ساختار کلی و آمار آن بررسی شد. داده‌ها به دو بخش آموزش و آزمون تقسیم شدند و ۱۰٪ از داده‌های آموزش برای تشکیل مجموعه اعتبارسنجی جدا شد. توکن‌سازی داده‌ها با روش ساده و مبتنی بر فاصله (Whitespace Tokenization) انجام شد.

۱-۱-۱ <PAD> و <UNK>

• **توکن <PAD>:** این توکن برای هم‌طول‌سازی جملات با طول متغیر استفاده می‌شود. از آن‌جا که مدل‌های RNN و LSTM نیاز به پردازش دسته‌ای (Batch) از داده‌ها دارند و طول ورودی‌ها باید یکنواخت باشد، جملات کوتاه‌تر با توکن <PAD> پر می‌شوند. این توکن نقش خنثی دارد و به مدل اطلاعات معنایی اضافه نمی‌کند؛ بنابراین هنگام محاسبه تابع هزینه، معمولاً برچسب‌های متناظر با <PAD> نادیده گرفته می‌شوند.

• **توکن <UNK>:** این توکن برای مدیریت کلمات ناشناخته در مجموعه‌داده آزمون یا اعتبارسنجی کاربرد دارد. اگر جمله‌ای شامل کلمه‌ای باشد که در واژگان آموزشی وجود ندارد، آن کلمه با <UNK> جایگزین می‌شود. این روش تضمین می‌کند که مدل بتواند با کلمات جدید یا کم‌تکرار مواجه شود بدون آنکه خطا در ورودی ایجاد شود، و همچنین از سقوط مدل به دلیل مواجهه با داده‌های خارج از واژگان جلوگیری می‌کند.

استفاده از این دو توکن باعث افزایش پایداری و کارایی آموزش مدل می‌شود و نقش اساسی در مدیریت داده‌های غیر همگن و ورودی‌های ناشناخته دارد.

در نهایت، کلاس‌های Dataset و DataLoader به‌صورت سفارشی پیاده‌سازی شد. برای مدیریت مؤثر جملات با طول متغیر، عمل *padding* به‌صورت دینامیک در سطح هر Batch با استفاده از یک تابع Collate انجام شد. این روش نسبت به هم‌طول‌سازی کل جملات بر اساس طول بیشینه مجموعه داده مزایای قابل توجهی دارد: کاهش محاسبات غیرضروری، صرفه‌جویی در حافظه، و افزایش سرعت و پایداری فرآیند آموزش مدل‌ها. این طراحی باعث می‌شود که مدل بتواند با داده‌های واقعی و جملات با طول متغیر به‌طور بهینه کار کند و از ایجاد خطاهای ناشی از پردازش داده‌های پر شده‌ی بیش از حد جلوگیری شود.

در این بخش، دو روش رایج برای هم‌طول‌سازی جملات نمایش داده می‌شود: **Padding Full** که جملات کوتاه به طول بزرگترین جمله در کل مجموعه داده پر می‌شوند، و **Padding Batch** که جملات تنها تا طول بزرگترین جمله در هر دسته پر می‌شوند. این دو روش تفاوت‌هایی در طول جملات ایجاد می‌کنند.

- **Full Padding Example:**

[illegible]

– **Decoded Tokens:** [‘BOS’, ‘us’, ‘ᠪᠠᠯᠲᠢᠮᠤᠷᠦ’, ‘baltimore’, ‘to’, ‘philadelphia’,
‘what’, ‘is’, ‘the’, ‘round’, ‘trip’, ‘fare’, ‘EOS’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’,
‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’,
‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’,
‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’,
‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’, ‘<PAD>’]

– Length: ୧୮

- **Batch Padding Example:**

- **Token IDs:** ۲، ۳، ۳۰، ۲۵، ۵۳۳، ۳۱، ۳۲، ۱۸، ۴۲، ۳۵۸، ۱۸، ۱۰۲، ۲۷، ۱۶، ۰، ۰، ۰، ۰، ۰، ۰
- **Decoded Tokens:** ['BOS', 'what', 'is', 'the', 'total', 'seating', 'capacity', 'of', 'all', 'aircraft', 'of', 'american', 'airlines', 'EOS', '<PAD>', '<PAD>', '<PAD>', '<PAD>', '<PAD>']
- **Length:** ۲۱

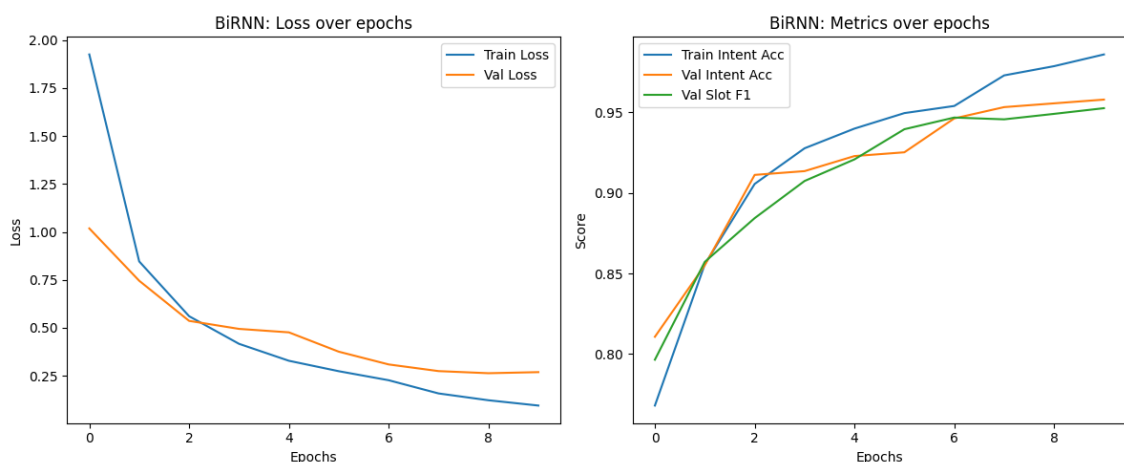
تفاوت در طول جملات: Padding Full جملات را تا طول بزرگترین جمله در کل مجموعه داده پر می‌کند (۴۸)، در حالی که Padding Batch تنها تا طول بزرگترین جمله در هر دسته پر می‌شود (۲۱). این روش دوم باعث کاهش مصرف حافظه و افزایش سرعت پردازش در آموزش می‌شود.

۲-۱ BiRNN

۱-۲-۱ معماری مدل BiRNN

این مدل طبق معماری تشریح شده در صورت سوال پیاده‌سازی شد و نتایج آن به شرح زیر است.

۲-۲-۱ نتایج آموزش



شکل ۱: نمودار آموزش BiRNN

جدول ۱: نتایج کلی BiRNN روی داده‌های تست

مقدار	معیار
۹۶.۲۵%	Accuracy Intent
۹۴.۷۹%	F1 Slot

۳-۲-۱ گزارش دسته‌بندی Intent

جدول ۲: BiRNN for Report Classification Intent

Intent	Precision	Recall	F1-score	Support
atis_city	۱.۰۰	۰.۳۳	۰.۵۰	۳
atis_flight_no	۱.۰۰	۰.۵۰	۰.۶۷	۲
atis_capacity	۱.۰۰	۱.۰۰	۱.۰۰	۴
atis_ground_fare	۱.۰۰	۰.۷۵	۰.۸۶	۴
atis_flight#atis_airfare	۱.۰۰	۰.۶۷	۰.۸۰	۳
atis_ground_service	۰.۹۴	۱.۰۰	۰.۹۷	۲۹
atis_distance	۱.۰۰	۱.۰۰	۱.۰۰	۳
atis_meal	۰.۰۰	۰.۰۰	۰.۰۰	۱
atis_flight	۰.۹۸	۰.۹۹	۰.۹۹	۴۲۴
atis_abbreviation	۰.۷۸	۰.۸۸	۰.۸۲	۱۶
atis_restriction	۰.۰۰	۰.۰۰	۰.۰۰	۱
atis_quantity	۰.۸۳	۱.۰۰	۰.۹۱	۵
atis_airline	۰.۸۱	۰.۹۴	۰.۸۷	۱۸
atis_aircraft	۱.۰۰	۰.۵۰	۰.۶۷	۸
atis_airport	۰.۶۰	۰.۷۵	۰.۶۷	۴
atis_airfare	۰.۹۸	۰.۹۸	۰.۹۸	۵۴
atis_flight_time	۱.۰۰	۰.۵۷	۰.۷۳	۷
Macro avg	۰.۸۲	۰.۷۰	۰.۷۳	۵۸۶
Weighted avg	۰.۹۶	۰.۹۶	۰.۹۶	۵۸۶

۴-۲-۱ گزارش دسته‌بندی Slot

longtable

جدول ۳: BiRNN for Report Classification Slot

Slot	Precision	Recall	F1-score	Support
aircraft_code	۰.۶۷	۰.۸۶	۰.۷۵	۷
airline_code	۱.۰۰	۰.۹۵	۰.۹۸	۲۲
airline_name	۰.۹۷	۰.۹۸	۰.۹۷	۹۱
airport_code	۰.۶۰	۰.۷۵	۰.۶۷	۴
airport_name	۰.۴۴	۰.۵۰	۰.۴۷	۸
arrive_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۱
arrive_date.day_name	۱.۰۰	۰.۴۰	۰.۵۷	۱۰
arrive_date.day_number	۱.۰۰	۰.۸۶	۰.۹۲	۷
arrive_date.month_name	۱.۰۰	۰.۸۶	۰.۹۲	۷
arrive_time.end_time	۱.۰۰	۰.۶۷	۰.۸۰	۳
arrive_time.period_mod	۰.۰۰	۰.۰۰	۰.۰۰	۱
arrive_time.period_of_day	۰.۶۰	۰.۳۸	۰.۴۶	۸
arrive_time.start_time	۱.۰۰	۰.۶۷	۰.۸۰	۳
arrive_time.time	۰.۷۱	۰.۷۱	۰.۷۱	۱۷
arrive_time.time_relative	۰.۹۳	۰.۷۲	۰.۸۱	۱۸
city_name	۰.۸۹	۰.۸۳	۰.۸۶	۳۰
class_type	۱.۰۰	۰.۹۶	۰.۹۸	۲۶
connect	۱.۰۰	۰.۸۰	۰.۸۹	۵
cost_relative	۱.۰۰	۱.۰۰	۱.۰۰	۳۸
day_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
depart_date.date_relative	۰.۸۰	۱.۰۰	۰.۸۹	۱۲
depart_date.day_name	۰.۹۴	۰.۹۹	۰.۹۶	۱۱۸

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۳

Slot	Precision	Recall	F1-score	Support
depart_date.day_number	۰.۹۳	۱.۰۰	۰.۹۷	۴۳
depart_date.month_name	۰.۹۵	۱.۰۰	۰.۹۸	۴۰
depart_date.today_relative	۰.۸۳	۰.۸۳	۰.۸۳	۱۲
depart_date.year	۰.۶۷	۰.۶۷	۰.۶۷	۳
depart_time.end_time	۱.۰۰	۱.۰۰	۱.۰۰	۳
depart_time.period_mod	۰.۷۱	۱.۰۰	۰.۸۳	۵
depart_time.period_of_day	۰.۹۳	۰.۹۹	۰.۹۶	۷۸
depart_time.start_time	۱.۰۰	۱.۰۰	۱.۰۰	۳
depart_time.time	۰.۸۶	۰.۹۵	۰.۹۰	۴۴
depart_time.time_relative	۰.۹۱	۱.۰۰	۰.۹۵	۴۱
economy	۱.۰۰	۱.۰۰	۱.۰۰	۴
fare_amount	۱.۰۰	۰.۸۰	۰.۸۹	۵
fare_basis_code	۰.۷۵	۱.۰۰	۰.۸۶	۶
flight_days	۱.۰۰	۱.۰۰	۱.۰۰	۶
flight_mod	۰.۸۹	۰.۸۹	۰.۸۹	۳۸
flight_number	۰.۹۰	۰.۸۲	۰.۸۶	۱۱
flight_stop	۱.۰۰	۱.۰۰	۱.۰۰	۱۹
flight_time	۰.۸۸	۰.۷۸	۰.۸۲	۹
fromloc.airport_code	۱.۰۰	۰.۳۳	۰.۵۰	۳
fromloc.airport_name	۰.۵۵	۰.۶۷	۰.۶۰	۹
fromloc.city_name	۰.۹۸	۰.۹۹	۰.۹۸	۴۹۹
fromloc.state_code	۱.۰۰	۰.۹۰	۰.۹۵	۱۰
fromloc.state_name	۱.۰۰	۰.۸۳	۰.۹۱	۶

بعد صفحه در ادامه

قبل صفحه از جدول ادامه – Table ۳

Slot	Precision	Recall	F ₁ -score	Support
meal	۱.۰۰	۱.۰۰	۱.۰۰	۷
meal_code	۱.۰۰	۰.۵۰	۰.۶۷	۲
meal_description	۱.۰۰	۰.۸۶	۰.۹۲	۷
mod	۱.۰۰	۰.۷۵	۰.۸۶	۴
or	۰.۹۲	۱.۰۰	۰.۹۶	۱۱
period_of_day	۰.۰۰	۰.۰۰	۰.۰۰	۲
restriction_code	۱.۰۰	۰.۷۵	۰.۸۶	۴
return_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۲
return_date.day_number	۰.۰۰	۰.۰۰	۰.۰۰	۱
return_date.month_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
round_trip	۰.۹۵	۰.۹۷	۰.۹۶	۳۸
state_code	۱.۰۰	۱.۰۰	۱.۰۰	۱
state_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
stoploc.city_name	۰.۹۳	۱.۰۰	۰.۹۶	۲۶
stoploc.state_code	۱.۰۰	۱.۰۰	۱.۰۰	۱
toloc.airport_code	۱.۰۰	۰.۵۰	۰.۶۷	۲
toloc.airport_name	۱.۰۰	۰.۸۰	۰.۸۹	۵
toloc.city_name	۰.۹۷	۰.۹۹	۰.۹۸	۴۹۷
toloc.country_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
toloc.state_code	۰.۹۰	۱.۰۰	۰.۹۵	۹
toloc.state_name	۰.۶۴	۰.۸۸	۰.۷۴	۸
transport_type	۰.۷۱	۰.۸۳	۰.۷۷	۶
micro avg	۰.۹۴	۰.۹۵	۰.۹۵	۱۹۷۰

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۳

Slot	Precision	Recall	F1-score	Support
macro avg	۰.۷۸	۰.۷۳	۰.۷۵	۱۹۷۰
weighted avg	۰.۹۴	۰.۹۵	۰.۹۴	۱۹۷۰

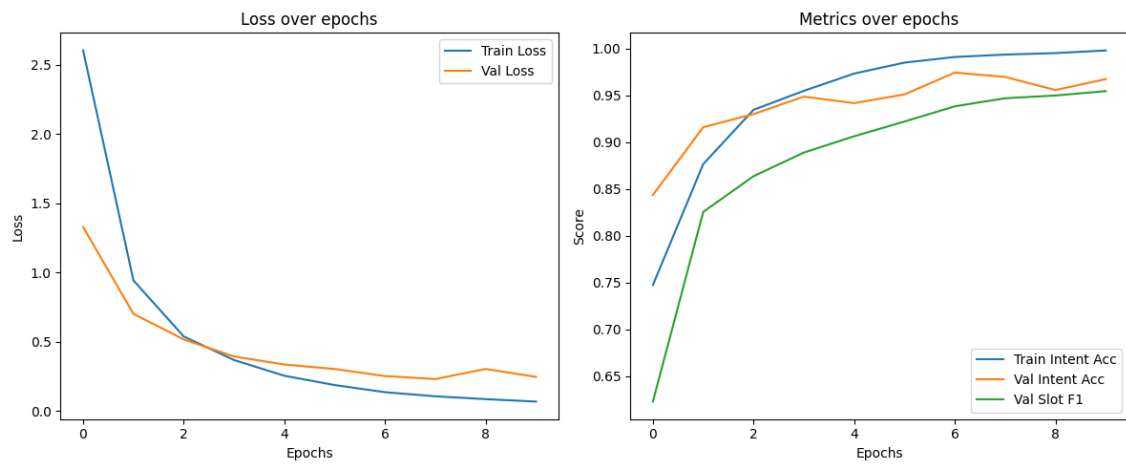
۵-۲-۱ تفسیر نتایج

- مدل BiRNN توانسته دقت بسیار بالایی در شناسایی Intent و های Slot پرتکرار داشته باشد.
- کلاس‌ها یا اسلات‌های کم‌نمونه، مانند `atis_city` یا `airport_code` دقت پایینی دارند، که طبیعی است.
- مدل BiRNN با استفاده از معماری دوطرفه و پردازش توالی‌ها، توانسته عملکرد بسیار مناسبی برای سیستم‌های SLU ارائه دهد. بهبود دقت روی کلاس‌های کم‌نمونه با افزایش داده‌ها یا تکنیک‌های Data Augmentation امکان‌پذیر است.

۳-۱ BiLSTM

مدل BiLSTM یک شبکه بازگشتی دوجهته است که برای تحلیل توالی‌ها مورد استفاده قرار می‌گیرد. خروجی‌های دو مسیر با هم ترکیب شده و باعث می‌شوند مدل بتواند اطلاعات گذشته و آینده‌ی توالی را همزمان درک کند. این ویژگی برای پردازش زبان طبیعی و به‌ویژه شناسایی اسلات‌ها و دسته‌بندی هدف بسیار مؤثر است. این مدل طبق معماری و هایپر پارامترهای توصیف شده در متن سوال پیاده‌سازی گردید و نتایج آن به شرح زیر است.

۱ مدل‌های RNN برای وظایف مشترک درک زبان گفتاری



شکل ۲: نمودار آموزش BiLSTM

۱-۳-۱ نتایج روی دیتای تست

جدول ۴: نتایج مدل BiLSTM روی دیتای تست

مقدار	معیار
۹۷.۴۴%	Intent Accuracy)
۹۴.۰۶%	Slot F1 Score

۲-۳-۱ گزارش دسته‌بندی هدف (Intent Classification)

جدول ۵: BiLSTM for Report Classification Intent

Intent	Precision	Recall	F1-score	Support
atis_city	۱.۰۰	۰.۶۷	۰.۸۰	۳
atis_flight_no	۱.۰۰	۱.۰۰	۱.۰۰	۲
atis_capacity	۱.۰۰	۱.۰۰	۱.۰۰	۴
atis_ground_fare	۰.۸۰	۱.۰۰	۰.۸۹	۴
atis_flight#atis_airfare	۰.۷۵	۱.۰۰	۰.۸۶	۳
atis_ground_service	۱.۰۰	۱.۰۰	۱.۰۰	۲۹
atis_distance	۱.۰۰	۰.۶۷	۰.۸۰	۳
atis_meal	۱.۰۰	۱.۰۰	۱.۰۰	۱
atis_flight	۰.۹۸	۰.۹۹	۰.۹۹	۴۲۴
atis_abbreviation	۰.۹۳	۰.۸۸	۰.۹۰	۱۶
atis_restriction	۰.۰۰	۰.۰۰	۰.۰۰	۱
atis_quantity	۰.۸۳	۱.۰۰	۰.۹۱	۵
atis_airline	۰.۸۶	۱.۰۰	۰.۹۲	۱۸
atis_aircraft	۰.۸۳	۰.۶۲	۰.۷۱	۸
atis_airport	۱.۰۰	۱.۰۰	۱.۰۰	۴
atis_airfare	۱.۰۰	۰.۹۸	۰.۹۹	۵۴
atis_flight_time	۱.۰۰	۰.۸۶	۰.۹۲	۷
macro avg	۰.۸۸	۰.۸۶	۰.۸۶	۵۸۶
weighted avg	۰.۹۷	۰.۹۷	۰.۹۷	۵۸۶

۳-۳-۱ Slot Classification

جدول ۶: BiLSTM for Report Classification Slot

Slot	Precision	Recall	F1-score	Support
aircraft_code	۱.۰۰	۰.۱۴	۰.۲۵	۷
airline_code	۰.۹۵	۰.۸۲	۰.۸۸	۲۲
airline_name	۰.۹۷	۰.۹۹	۰.۹۸	۹۱
airport_code	۰.۰۰	۰.۰۰	۰.۰۰	۴
airport_name	۰.۵۷	۰.۵۰	۰.۵۳	۸

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۶

Slot	Precision	Recall	F ₁ -score	Support
arrive_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۱
arrive_date.day_name	۱.۰۰	۰.۵۰	۰.۶۷	۱۰
arrive_date.day_number	۰.۸۰	۰.۵۷	۰.۶۷	۷
arrive_date.month_name	۰.۸۳	۰.۷۱	۰.۷۷	۷
arrive_time.end_time	۰.۰۰	۰.۰۰	۰.۰۰	۳
arrive_time.period_mod	۰.۰۰	۰.۰۰	۰.۰۰	۱
arrive_time.period_of_day	۰.۷۱	۰.۶۲	۰.۶۷	۸
arrive_time.start_time	۱.۰۰	۱.۰۰	۱.۰۰	۳
arrive_time.time	۰.۵۹	۰.۷۶	۰.۶۷	۱۷
arrive_time.time_relative	۰.۹۳	۰.۷۲	۰.۸۱	۱۸
city_name	۰.۸۴	۰.۸۷	۰.۸۵	۳۰
class_type	۱.۰۰	۰.۹۶	۰.۹۸	۲۶
connect	۱.۰۰	۰.۸۰	۰.۸۹	۵
cost_relative	۰.۹۷	۰.۹۷	۰.۹۷	۳۸
day_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
depart_date.date_relative	۰.۹۲	۱.۰۰	۰.۹۶	۱۲
depart_date.day_name	۰.۹۵	۰.۹۹	۰.۹۷	۱۱۸
depart_date.day_number	۰.۸۵	۰.۹۵	۰.۹۰	۴۳
depart_date.month_name	۰.۹۵	۰.۹۷	۰.۹۶	۴۰
depart_date.today_relative	۰.۸۳	۰.۸۳	۰.۸۳	۱۲
depart_date.year	۱.۰۰	۰.۶۷	۰.۸۰	۳
depart_time.end_time	۰.۶۷	۰.۶۷	۰.۶۷	۳
depart_time.period_mod	۰.۶۲	۱.۰۰	۰.۷۷	۵

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۶

Slot	Precision	Recall	F1-score	Support
depart_time.period_of_day	۰.۹۶	۰.۹۵	۰.۹۵	۷۸
depart_time.start_time	۱.۰۰	۰.۶۷	۰.۸۰	۳
depart_time.time	۰.۸۷	۰.۸۹	۰.۸۸	۴۴
depart_time.time_relative	۰.۹۵	۰.۹۸	۰.۹۶	۴۱
economy	۱.۰۰	۱.۰۰	۱.۰۰	۴
fare_amount	۱.۰۰	۰.۸۰	۰.۸۹	۵
fare_basis_code	۰.۸۳	۰.۸۳	۰.۸۳	۶
flight_days	۱.۰۰	۱.۰۰	۱.۰۰	۶
flight_mod	۰.۹۷	۰.۸۷	۰.۹۲	۳۸
flight_number	۰.۵۰	۰.۱۸	۰.۲۷	۱۱
flight_stop	۱.۰۰	۱.۰۰	۱.۰۰	۱۹
flight_time	۰.۸۸	۰.۷۸	۰.۸۲	۹
fromloc.airport_code	۱.۰۰	۰.۳۳	۰.۵۰	۳
fromloc.airport_name	۰.۳۰	۰.۳۳	۰.۳۲	۹
fromloc.city_name	۰.۹۸	۰.۹۹	۰.۹۹	۴۹۹
fromloc.state_code	۱.۰۰	۰.۹۰	۰.۹۵	۱۰
fromloc.state_name	۱.۰۰	۰.۸۳	۰.۹۱	۶
meal	۱.۰۰	۱.۰۰	۱.۰۰	۷
meal_code	۰.۰۰	۰.۰۰	۰.۰۰	۲
meal_description	۱.۰۰	۰.۷۱	۰.۸۳	۷
mod	۱.۰۰	۰.۵۰	۰.۶۷	۴
or	۰.۹۱	۰.۹۱	۰.۹۱	۱۱
period_of_day	۰.۰۰	۰.۰۰	۰.۰۰	۲

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۶

Slot	Precision	Recall	F ₁ -score	Support
restriction_code	۰.۵۰	۰.۲۵	۰.۳۳	۴
return_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۲
return_date.day_number	۰.۰۰	۰.۰۰	۰.۰۰	۱
return_date.month_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
round_trip	۰.۹۷	۰.۹۷	۰.۹۷	۳۸
state_code	۱.۰۰	۱.۰۰	۱.۰۰	۱
state_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
stoploc.city_name	۰.۹۳	۰.۹۶	۰.۹۴	۲۶
stoploc.state_code	۱.۰۰	۱.۰۰	۱.۰۰	۱
toloc.airport_code	۱.۰۰	۰.۵۰	۰.۶۷	۲
toloc.airport_name	۰.۷۵	۰.۶۰	۰.۶۷	۵
toloc.city_name	۰.۹۹	۰.۹۹	۰.۹۹	۴۹۷
toloc.country_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
toloc.state_code	۰.۹۰	۱.۰۰	۰.۹۵	۹
toloc.state_name	۰.۸۰	۱.۰۰	۰.۸۹	۸
transport_type	۰.۸۳	۰.۸۳	۰.۸۳	۶
micro avg	۰.۹۵	۰.۹۳	۰.۹۴	۱۹۷۰
macro avg	۰.۷۳	۰.۶۵	۰.۶۷	۱۹۷۰
weighted avg	۰.۹۴	۰.۹۳	۰.۹۳	۱۹۷۰

۴-۳-۱ تفسیر نتایج

- دقت بسیار بالا در دسته‌بندی Intent نشان‌دهنده‌ی توانایی مدل در شناسایی نوع درخواست کاربر است.
- Score ۱F اسلات‌ها بیانگر عملکرد مناسب مدل در استخراج موجودیت‌ها از جملات است، اما برخی

اسلات‌های کم‌نمونه مانند `return_date.day_number` صفر شده‌اند که ناشی از کمبود نمونه در دیتاست است.

- مدل توانسته است به خوبی نام شهرها و کلاس پروازها را تشخیص دهد (`fromloc.city_name`، `toloc.city_name` با ۱F نزدیک ۹۹.۰).
- اسلات‌هایی که Precision یا Recall صفر دارند، نیازمند داده‌های بیشتر برای آموزش هستند.

۴-۱ Decoder Encoder

در این بخش، یک مدل **Encoder-Decoder غیرهم‌تراز (Non-aligned)** برای انجام همزمان دو وظیفه‌ی **تشخیص نیت (Intent Detection)** و **استخراج اسلات (Slot Filling)** مورد بررسی قرار گرفته است. برخلاف مدل‌های پیشین مانند BiRNN و BiLSTM که برچسب اسلات را به‌صورت مستقیم برای هر توکن ورودی پیش‌بینی می‌کردند، در این معماری، مسئله‌ی slot filling به‌صورت یک **مسئله‌ی تولید توالی (Se-quence Generation)** مدل‌سازی شده است. در ادامه، ساختار مدل، نتایج تجربی و تحلیل دقیق نقاط قوت و ضعف آن ارائه می‌شود.

۱-۴-۱ تفاوت بنیادین معماری BiLSTM و Encoder-Decoder

مدل‌های BiLSTM مورد استفاده در بخش‌های قبلی، بر پایه‌ی یک اصل ساده بنا شده‌اند: هر توکن ورودی دقیقاً با یک برچسب اسلات متناظر است. در این مدل‌ها، خروجی شبکه در هر گام زمانی، مستقیماً برچسب slot همان توکن را پیش‌بینی می‌کند. این نوع معماری را می‌توان یک **مدل هم‌تراز (Aligned)** نامید، زیرا بین ورودی و خروجی، یک تناظر یک‌به‌یک صریح وجود دارد.

در مقابل، معماری Encoder-Decoder دارای ماهیتی کاملاً متفاوت است. در این مدل، ابتدا Encoder کل جمله‌ی ورودی را به یک نمایش فشرده در فضای نهان تبدیل می‌کند. سپس Decoder با استفاده از این نمایش فشرده، خروجی را به‌صورت یک توالی جدید تولید می‌کند. در نتیجه، در این چارچوب، ها slot نه به‌عنوان برچسب توکن‌ها، بلکه به‌عنوان یک **جمله‌ی مستقل** در نظر گرفته می‌شوند که باید تولید شود. بنابراین، تفاوت بنیادین این دو معماری در این نکته خلاصه می‌شود که:

• BiLSTM یک مدل برچسب گذاری توکن محور است،

• در حالی که Encoder-Decoder یک مدل تولیدکننده ی توالی محسوب می شود.

این تفاوت، هم ظرفیت های جدیدی به مدل اضافه می کند و هم چالش های جدیدی را به همراه دارد.

۱-۴-۲ نقش Decoder و اثر افزایش عمق آن در مدل

افزودن Decoder به معماری شبکه، قابلیت های مهمی را در اختیار مدل قرار می دهد. Decoder این امکان را فراهم می کند که خروجی ها به صورت وابسته به یکدیگر و با در نظر گرفتن تاریخچه ی تولید شوند. به بیان دقیق تر، هر اسلات تولید شده، نه تنها به خروجی Encoder، بلکه به اسلات های قبلی نیز وابسته است.

افزایش تعداد لایه های Decoder باعث می شود مدل بتواند:

۱. وابستگی های بلندمدت بین اسلات ها را بهتر مدل کند؛ برای مثال، الگوی تکرار شونده ی حضور city_name قبل از date یا time.

۲. سطوح مختلفی از انتزاع را بیاموزد؛ لایه های پایین تر اطلاعات محلی و لغوی را پردازش می کنند، در حالی که لایه های بالاتر روابط معنایی و ساختاری میان اسلات ها را استخراج می کنند.

۳. نوعی گرامر ضمنی برای توالی اسلات ها یاد بگیرد، به گونه ای که ترتیب منطقی اسلات ها در خروجی حفظ شود.

با این حال، افزایش عمق Decoder هزینه هایی نیز به همراه دارد، از جمله دشوارتر شدن فرایند آموزش، افزایش حساسیت به داده های آموزشی و تشدید پدیده ی خطای انباشتی در زمان decoding.

۱-۴-۳ ضرورت استفاده از توکن های <BOS> و <EOS>

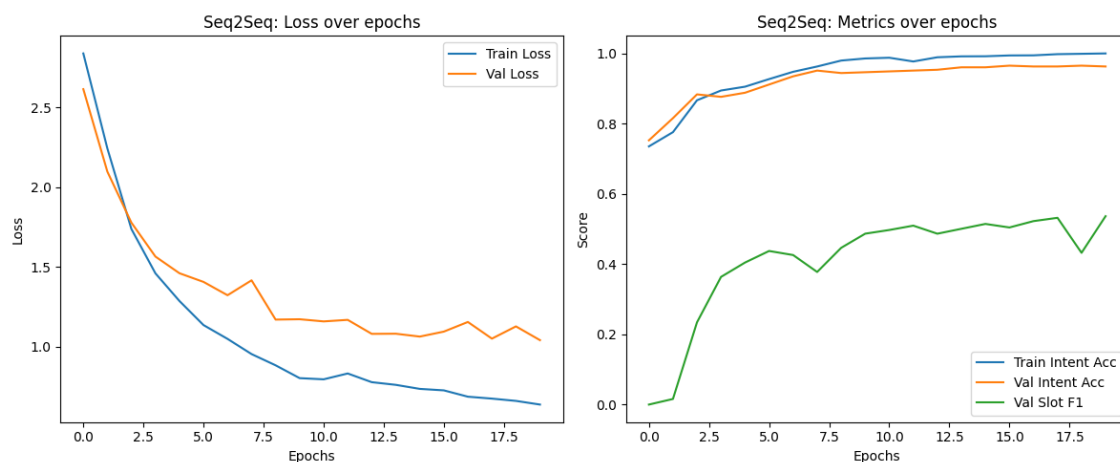
از آنجا که Decoder یک مولد توالی است، لازم است نقطه ی شروع و پایان فرایند تولید به طور صریح مشخص شود. توکن <BOS> به Decoder اعلام می کند که فرایند تولید خروجی آغاز شده است و اولین اسلات باید پس از آن تولید شود. از سوی دیگر، توکن <EOS> نقش سیگنال توقف را ایفا می کند و به مدل می فهماند که تولید توالی به پایان رسیده است.

در نبود این توکن ها:

- مدل قادر به تشخیص طول مناسب خروجی نخواهد بود،
- فرآیند decoding ممکن است زودتر یا دیرتر از حد لازم متوقف شود،
- و همترازی لازم برای محاسبه‌ی تابع خطا از بین می‌رود.

بنابراین، وجود این دو توکن برای هر مدل Encoder-Decoder که خروجی متغیر طول تولید می‌کند، امری ضروری است.

۴-۴-۱ نتایج آموزش و ارزیابی عملکرد مدل روی داده‌ی آزمون



شکل ۳: نمودار آموزش Encoder-Decoder

جدول ۷: نتایج آزمایش مدل SEQ²SEQ

نتایج کلی مدل SEQ ² SEQ	
مقدار	معیار ارزیابی
%96.42	دقت کلی تشخیص نیت (Intent Accuracy)
%54.27	امتیاز F1 کلی اسلات‌ها (Slot F1 Score)

جدول ۸: Report Classification Intent

کلاس	Precision	Recall	F1-Score	Support
atis_abbreviation	۰.۸۸	۰.۸۸	۰.۸۸	۱۶
atis_aircraft	۰.۷۵	۰.۷۵	۰.۷۵	۸
atis_airfare	۰.۹۶	۰.۹۶	۰.۹۶	۵۴
atis_airline	۰.۹۰	۱.۰۰	۰.۹۵	۱۸
atis_airport	۱.۰۰	۰.۷۵	۰.۸۶	۴
atis_capacity	۱.۰۰	۱.۰۰	۱.۰۰	۴
atis_city	۱.۰۰	۰.۶۷	۰.۸۰	۳
atis_distance	۱.۰۰	۱.۰۰	۱.۰۰	۳
atis_flight	۰.۹۹	۰.۹۹	۰.۹۹	۴۲۴
atis_flight#atis_airfare	۰.۴۰	۰.۶۷	۰.۵۰	۳
atis_flight_no	۱.۰۰	۰.۵۰	۰.۶۷	۲
atis_flight_time	۱.۰۰	۰.۵۷	۰.۷۳	۷
atis_ground_fare	۱.۰۰	۰.۷۵	۰.۸۶	۴
atis_ground_service	۰.۹۷	۰.۹۷	۰.۹۷	۲۹
atis_meal	۰.۲۵	۱.۰۰	۰.۴۰	۱
atis_quantity	۰.۶۷	۰.۸۰	۰.۷۳	۵
atis_restriction	۰.۰۰	۰.۰۰	۰.۰۰	۱
Accuracy			۰.۹۶	
Macro Avg	۰.۸۱	۰.۷۸	۰.۷۷	۵۸۶
Weighted Avg	۰.۹۷	۰.۹۶	۰.۹۶	۵۸۶

جدول ۹: گزارش طبقه‌بندی اسلات‌ها (Slot Classification Report)

اسلات	دقت	فراخوانی	F1-score	نمونه تعداد
UNK>	۰.۰۰	۰.۰۰	۰.۰۰	۲
aircraft_code	۰.۰۰	۰.۰۰	۰.۰۰	۷
airline_code	۰.۶۰	۰.۱۴	۰.۲۲	۲۲

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۹

اسلات	دقت	فراخوانی	F1-score	نمونه تعداد
airline_name	۰.۰۶	۰.۰۲	۰.۰۳	۹۱
airport_code	۱.۰۰	۰.۵۰	۰.۶۷	۴
airport_name	۰.۰۰	۰.۰۰	۰.۰۰	۸
arrive_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۱
arrive_date.day_name	۰.۰۰	۰.۰۰	۰.۰۰	۱۰
arrive_date.day_number	۰.۰۰	۰.۰۰	۰.۰۰	۷
arrive_date.month_name	۰.۰۰	۰.۰۰	۰.۰۰	۷
arrive_time.end_time	۰.۰۰	۰.۰۰	۰.۰۰	۳
arrive_time.period_mod	۰.۰۰	۰.۰۰	۰.۰۰	۱
arrive_time.period_of_day	۰.۰۰	۰.۰۰	۰.۰۰	۸
arrive_time.start_time	۰.۰۰	۰.۰۰	۰.۰۰	۳
arrive_time.time	۰.۰۰	۰.۰۰	۰.۰۰	۱۷
arrive_time.time_relative	۰.۰۰	۰.۰۰	۰.۰۰	۱۸
city_name	۰.۵۵	۰.۵۳	۰.۵۴	۳۰
class_type	۰.۰۰	۰.۰۰	۰.۰۰	۲۶
connect	۰.۰۰	۰.۰۰	۰.۰۰	۵
cost_relative	۰.۷۱	۰.۲۶	۰.۳۸	۳۸
day_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
depart_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۱۲
depart_date.day_name	۰.۴۴	۰.۲۶	۰.۳۳	۱۱۸
depart_date.day_number	۰.۱۵	۰.۰۵	۰.۰۷	۴۳
depart_date.month_name	۰.۰۰	۰.۰۰	۰.۰۰	۴۰
depart_date.today_relative	۰.۰۰	۰.۰۰	۰.۰۰	۱۲

بعد صفحه در ادامه

قبل صفحه از جدول ادامه - Table ۹

اسلات	دقت	فراخوانی	F1-score	نمونه تعداد
depart_date.year	۰.۰۰	۰.۰۰	۰.۰۰	۳
depart_time.end_time	۰.۰۰	۰.۰۰	۰.۰۰	۳
depart_time.period_mod	۰.۰۰	۰.۰۰	۰.۰۰	۵
depart_time.period_of_day	۰.۱۵	۰.۱۴	۰.۱۵	۷۸
depart_time.start_time	۰.۰۰	۰.۰۰	۰.۰۰	۳
depart_time.time	۰.۰۰	۰.۰۰	۰.۰۰	۴۴
depart_time.time_relative	۰.۰۰	۰.۰۰	۰.۰۰	۴۱
economy	۰.۰۰	۰.۰۰	۰.۰۰	۴
fare_amount	۰.۰۰	۰.۰۰	۰.۰۰	۵
fare_basis_code	۱.۰۰	۰.۸۳	۰.۹۱	۶
flight_days	۰.۰۰	۰.۰۰	۰.۰۰	۶
flight_mod	۰.۰۰	۰.۰۰	۰.۰۰	۳۸
flight_number	۰.۰۰	۰.۰۰	۰.۰۰	۱۱
flight_stop	۰.۰۰	۰.۰۰	۰.۰۰	۱۹
flight_time	۰.۰۰	۰.۰۰	۰.۰۰	۹
fromloc.airport_code	۰.۰۰	۰.۰۰	۰.۰۰	۳
fromloc.airport_name	۰.۰۰	۰.۰۰	۰.۰۰	۹
fromloc.city_name	۰.۸۵	۰.۸۶	۰.۸۶	۴۹۹
fromloc.state_code	۰.۰۰	۰.۰۰	۰.۰۰	۱۰
fromloc.state_name	۰.۰۰	۰.۰۰	۰.۰۰	۶
meal	۰.۰۰	۰.۰۰	۰.۰۰	۷
meal_code	۰.۰۰	۰.۰۰	۰.۰۰	۲
meal_description	۰.۰۰	۰.۰۰	۰.۰۰	۷

بعد صفحه در ادامه

قبل صفحه از جدول ادامه – Table ۹

اسلات	دقت	فراخوانی	F1-score	نمونه تعداد
mod	۰.۰۰	۰.۰۰	۰.۰۰	۴
or	۰.۰۰	۰.۰۰	۰.۰۰	۱۱
period_of_day	۰.۰۰	۰.۰۰	۰.۰۰	۲
restriction_code	۰.۶۷	۰.۵۰	۰.۵۷	۴
return_date.date_relative	۰.۰۰	۰.۰۰	۰.۰۰	۲
round_trip	۰.۳۹	۰.۳۴	۰.۳۷	۳۸
state_code	۰.۰۰	۰.۰۰	۰.۰۰	۱
state_name	۰.۰۰	۰.۰۰	۰.۰۰	۱
stoploc.city_name	۰.۳۲	۰.۴۶	۰.۳۸	۲۶
stoploc.state_code	۰.۰۰	۰.۰۰	۰.۰۰	۱
toloc.airport_code	۰.۰۰	۰.۰۰	۰.۰۰	۲
toloc.airport_name	۰.۰۰	۰.۰۰	۰.۰۰	۵
toloc.city_name	۰.۷۱	۰.۷۱	۰.۷۱	۴۹۷
toloc.state_code	۰.۰۰	۰.۰۰	۰.۰۰	۹
toloc.state_name	۰.۰۰	۰.۰۰	۰.۰۰	۸
transport_type	۰.۰۰	۰.۰۰	۰.۰۰	۶
micro avg	۰.۶۸	۰.۴۵	۰.۵۴	۱۹۶۹
macro avg	۰.۱۲	۰.۰۹	۰.۱۰	۱۹۶۹
weighted avg	۰.۴۸	۰.۴۵	۰.۴۶	۱۹۶۹

نتایج تجربی نشان می‌دهد که مدل Encoder–Decoder در تشخیص intent عملکرد بسیار مطلوبی دارد و دقت آن به حدود ۹۶٪ رسیده است. این نتیجه نشان می‌دهد که Encoder قادر است نمایش معنایی مناسبی از کل جمله استخراج کند.

در مقابل، عملکرد مدل در وظیفه‌ی filling slot به‌طور محسوسی ضعیف‌تر از مدل‌های BiL- و BiRNN

STM است. مقدار ۱F برای اسلات‌ها اندکی بالاتر از آستانه‌ی قبولی (۵۰٪) بوده و در بسیاری از اسلات‌های کم‌نمونه، مدل عملاً ناتوان از پیش‌بینی صحیح است.

۵-۴-۱ تحلیل علت ضعف مدل در تشخیص Slot

ضعف عملکرد این مدل در filling slot را می‌توان به چند عامل اصلی نسبت داد:

۱. **عدم هم‌ترازی مستقیم:** این معماری فاقد هم‌ترازی مستقیم بین توکن‌های ورودی و اسلات‌های خروجی است. Decoder بدون دانستن این‌که هر اسلات به کدام توکن تعلق دارد، مجبور به تولید کل توالی می‌شود.

۲. **خطای انباشتی (Exposure Bias):** مدل دچار پدیده‌ی خطای انباشتی است؛ بدین معنا که یک خطای کوچک در ابتدای decoding می‌تواند موجب انحراف کل توالی خروجی شود.

۳. **عدم وجود مکانیزم توجه (Attention):** در این پیاده‌سازی از مکانیزم توجه (Attention) استفاده نشده است. در نتیجه، Decoder نمی‌تواند در هر گام روی بخش مرتبطی از ورودی تمرکز کند و ناچار است تنها به بردار فشرده‌ی Encoder تکیه کند.

۴. **عدم تحمیل صریح ساختار BIO:** ساختار خاص برچسب‌های اسلات (مانند قالب BIO) به‌طور صریح به مدل تحمیل نشده است و این موضوع باعث کاهش انسجام خروجی می‌شود.

۶-۴-۱ روش پیشنهادی مقاله برای رفع این مشکل

مطابق مقاله‌ی مرجع، یکی از راهکارهای پیشنهادی برای رفع مشکل ضعف slot filling در مدل‌های Encoder-Decoder غیر هم‌تراز، استفاده از Aligned Slot Generation یا اعمال نوعی محدودیت هم‌ترازی (Alignment Constraint) در فرآیند تولید است.

در مقاله اشاره می‌شود که مشکل اصلی این مدل‌ها، آزادی بیش از حد Decoder در تولید اسلات‌هاست، بدون آنکه موقعیت دقیق توکن‌های ورودی در نظر گرفته شود. برای حل این مسئله، نویسندگان پیشنهاد می‌کنند که به‌نحوی، ارتباط مکانی میان توکن‌های ورودی و اسلات‌های خروجی به مدل القا شود. این کار می‌تواند از طریق اعمال محدودیت در decoding یا افزودن سیگنال‌های کمکی برای موقعیت انجام شود.

به بیان دیگر، هدف این روش آن است که Decoder مجبور شود ساختار جمله‌ی ورودی را رعایت کند و از تولید اسلات‌های نامرتب یا جابه‌جا شده جلوگیری شود.

جمع‌بندی

در مجموع، مدل Encoder–Decoder Non-aligned توانایی بالایی در تشخیص intent نشان می‌دهد، اما به دلیل ماهیت تولیدی و عدم هم‌ترازی صریح، در وظیفه‌ی filling slot نسبت به مدل‌های BiLSTM عملکرد ضعیف‌تری دارد. این نتایج نشان می‌دهد که اگرچه معماری‌های Seq²Seq از نظر مفهومی قدرتمندتر هستند، اما برای وظایف توکن‌محور مانند slot، filling، نیازمند مکانیزم‌های تکمیلی مانند attention یا alignment constraint می‌باشند.

۱-۵ تحلیل مقایسه‌ای عملکرد مدل‌ها در Slot Filling

۱. مقایسه‌ی کیفی عملکرد سه مدل روی Slot Filling (جدول خلاصه)

ابتدا برای شفافیت، یک جدول خلاصه‌ی رفتار کلی سه مدل بر اساس ۵ نمونه‌ی تست ارائه می‌شود:

جدول ۱۰: مقایسه‌ی کیفی رفتار سه مدل در Filling Slot

مدل	دقت Intent	رفتار کلی Filling Slot	پایداری برچسب‌ها	وابستگی به موقعیت توکن
BiRNN	ضعیف	بسیار پرنویز	بسیار پایین	ضعیف
BiLSTM	خوب	نسبتاً دقیق	بالا	قوی
Seq ² Seq	خوب	بسیار محافظه‌کارانه	پایین	بسیار ضعیف

مشاهده می‌شود که Seq²Seq اگرچه intent را اغلب درست تشخیص می‌دهد، اما در filling slot یا همه‌چیز را 0 پیش‌بینی می‌کند یا اسلات‌ها را جابه‌جا تولید می‌کند.

۱-۵-۱ تحلیل دقیق خطاهای Seq²Seq با مثال‌های واقعی Greedy Decoding

مثال ۱

جمله:

please give me direct morning flights from pittsburgh to atlanta

برچسب‌های مهم واقعی:

B-connect → direct •

B-depart_time.period_of_day → morning •

B-fromloc.city_name → pittsburgh •

B-toloc.city_name → atlanta •

خروجی Seq:۲Seq

B-fromloc.city_name → pittsburgh فقط •

• تمام سایر توکن‌ها → 0

تحلیل: Decoder فقط یک اسلات پرتکرار و بسیار غالب (fromloc.city_name) را یاد گرفته و سایر اسلات‌ها را حذف کرده است. این نشان می‌دهد که مدل:

• وابستگی محلی بین *morning* و مفهوم زمان را از دست داده

• و ترتیب معنایی جمله را در تولید slot‌ها لحاظ نکرده است

مثال ۲

جمله:

flight times ... in the morning before 8

برچسب‌های واقعی:

B/I-flight_time → times flight •

B-depart_time.period_of_day → morning •

B-depart_time.time_relative → before •

B-depart_time.time → 8 •

خروجی Seq:۲Seq

• تقریباً تمام توکن‌ها → 0

• یک خطای شدید: B-stoploc.city_name → 8

تحلیل: این مثال به‌وضوح فقدان هم‌ترازی (alignment) را نشان می‌دهد:

• Decoder نمی‌داند که «عدد ۸» از نظر مکانی به زمان وابسته است

• چون slot‌ها را مستقل از توکن ورودی و صرفاً به‌عنوان یک توالی تولید می‌کند

مثال ۳

جمله کوتاه:

what is phl

برچسب واقعی:

B-airport_code → phl •

خروجی Seq:۲Seq

• همه چیز → 0

تحلیل: این نمونه به‌شدت گویاست:

• BiRNN و BiLSTM هرچند intent را اشتباه زده‌اند، اما حداقل تلاش کرده‌اند برچسب بدهند

• Seq:۲Seq به‌دلیل کوتاه بودن جمله و نبود سرنخ‌های کافی در decoding به‌طور کامل slot را حذف

کرده است

این نشان‌دهنده‌ی bias شدید مدل به سمت برچسب O است.

مثال ۴

جمله:

list flights from washington dc to pittsburgh

برچسب‌های واقعی:

B-fromloc.city_name → washington •

B-fromloc.state_code → dc •

B-toloc.city_name → pittsburgh •

خروجی Seq^۲Seq

(درست) B-fromloc.city_name → washington •

0 → dc • (غلط)

0 → pittsburgh • (غلط)

B-toloc.city_name → to • (کاملاً جابه‌جا)

تحلیل: اینجا مشکل اصلی Seq^۲Seq کاملاً مشخص است:

- مدل اسلات را در موقعیت اشتباه تولید می‌کند

- یعنی slot درست را دارد، اما به توکن اشتباه نسبت می‌دهد

این دقیقاً همان مشکلی است که در مدل‌های Non-aligned انتظار می‌رود.

مثال ۵

جمله:

what airlines from washington dc to columbus

برچسب‌های واقعی:

B-fromloc.city_name → washington •

B-fromloc.state_code → dc •

B-toloc.city_name → columbus •

خروجی Seq^۲Seq

B-fromloc.city_name → dc • (جابجایی)

B-toloc.city_name → columbus • (درست)

تحلیل: Decoder موفق شده intent را درست تشخیص دهد، اما:

• city و state را قاطعی کرده

• نشان می‌دهد که ساختار BIO و وابستگی محلی توکن‌ها رعایت نمی‌شود

۳. دلایل اصلی ضعف Seq^۲Seq در Filling Slot (جمع‌بندی تحلیلی)

با اتکا به این خروجی‌ها، دلایل ضعف مدل Seq^۲Seq به‌صورت دقیق به شرح زیر است:

۱ نبود هم‌ترازی مستقیم بین ورودی و خروجی

در BiLSTM، هر توکن دقیقاً یک خروجی دارد. در Seq^۲Seq، slot‌ها به‌عنوان جمله‌ای مستقل تولید

می‌شوند. □ نتیجه: جابه‌جایی، حذف و نسبت دادن slot به توکن اشتباه.

۲ خطای انباشتی در Greedy Decoding

در decoding حریصانه:

- اگر یک slot اشتباه تولید شود
- تمام توالی بعدی منحرف می‌شود

این اثر در مثال‌های ۲ و ۴ کاملاً مشهود است.

۳ سوگیری شدید به برچسب O

در گزارش sequeval دیدیم:

- ۱F macro بسیار پایین
- ۱F weighted متوسط

این یعنی:

مدل ترجیح می‌دهد چیزی پیش‌بینی نکند تا اشتباه کند

۴ عدم استفاده از اطلاعات محلی Local Context

BiLSTM از چپ و راست هر توکن را می‌بیند. Seq²Seq فقط یک بردار فشرده از کل جمله دارد. □
مفاهیمی مثل *before*، *morning* □ که به موقعیت وابسته‌اند، از بین می‌روند.

۴. نتیجه‌گیری نهایی

با استناد مستقیم به خروجی‌های تست، می‌توان نتیجه گرفت که:

- مدل Encoder–Decoder در **تشخیص intent بسیار موفق** است، زیرا intent به یک نمایش کلی از جمله وابسته است.

• اما در **filling slot** به‌طور ساختاری نامناسب است، زیرا:

۱. هم‌ترازی توکن-اسلات ندارد

۲. decoding آن حساس به خطاست

۳. و ساختار BIO را به‌صورت صریح مدل نمی‌کند

به همین دلیل، علی‌رغم پیچیدگی بیشتر، این معماری در وظیفه‌ی **filling slot** ضعیف‌تر از **BiLSTM** عمل می‌کند.

۲ پیش‌بینی قیمت سهام با استفاده از مدل‌های Recursive

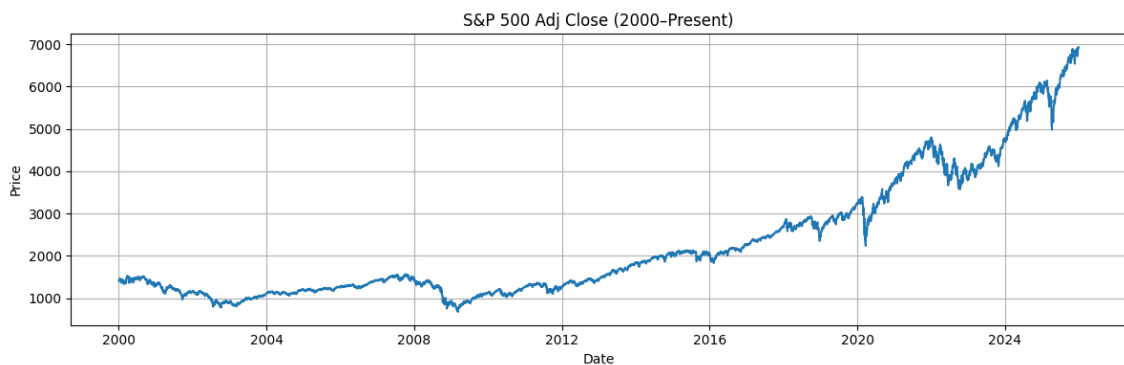
۱-۲ پیش‌پردازش داده‌ها

۱-۱-۲

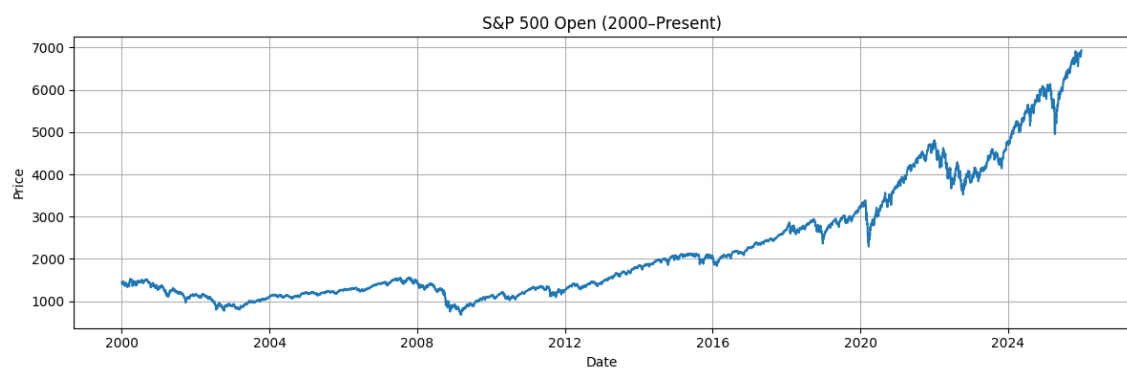
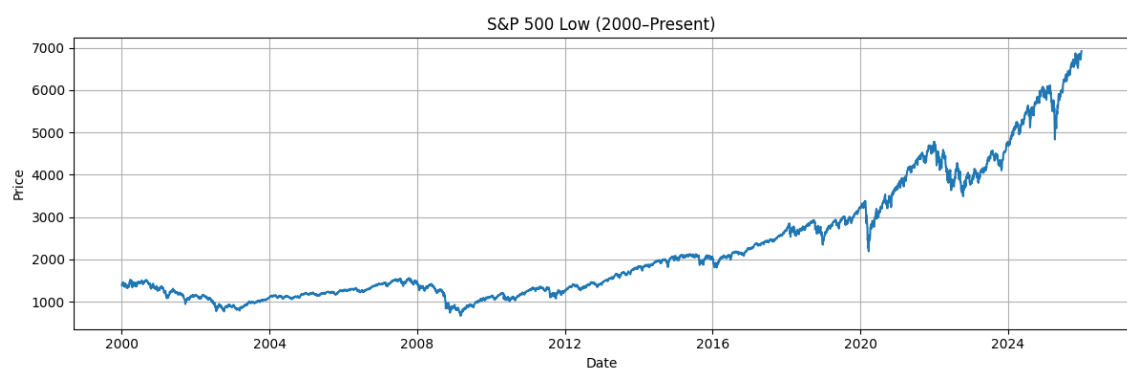
چه ویژگی‌هایی در این مجموعه داده وجود دارد؟

['Date', 'Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume']

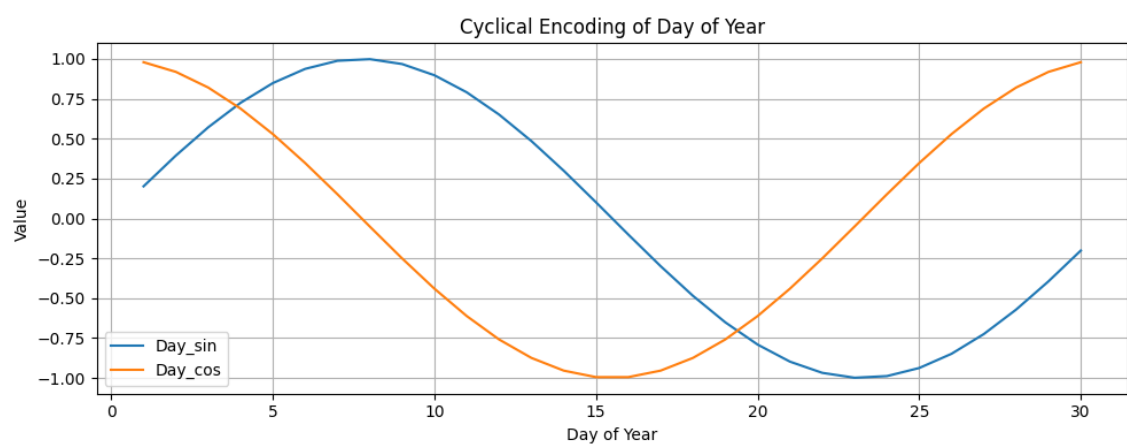
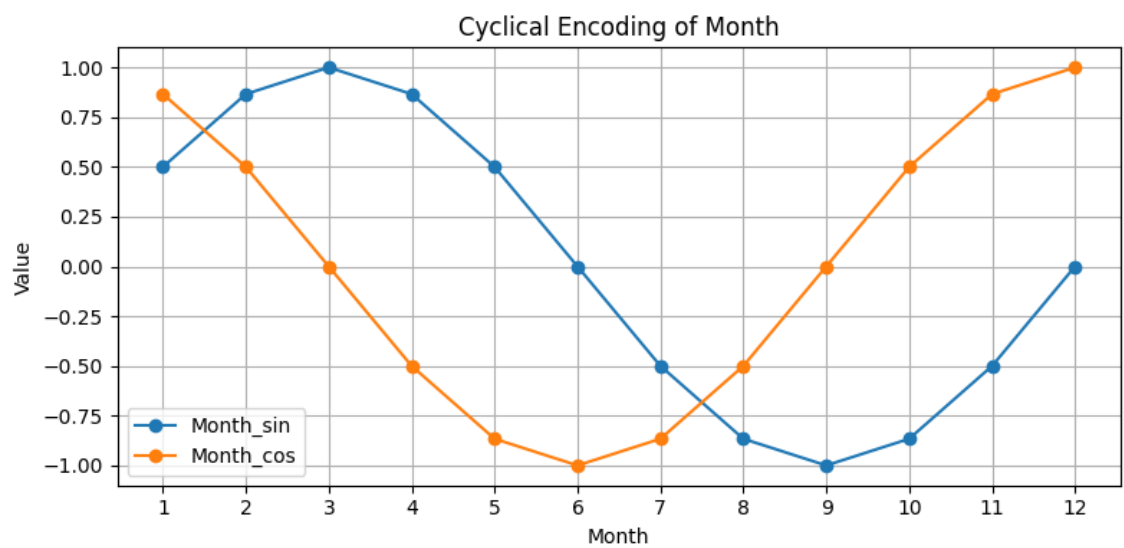
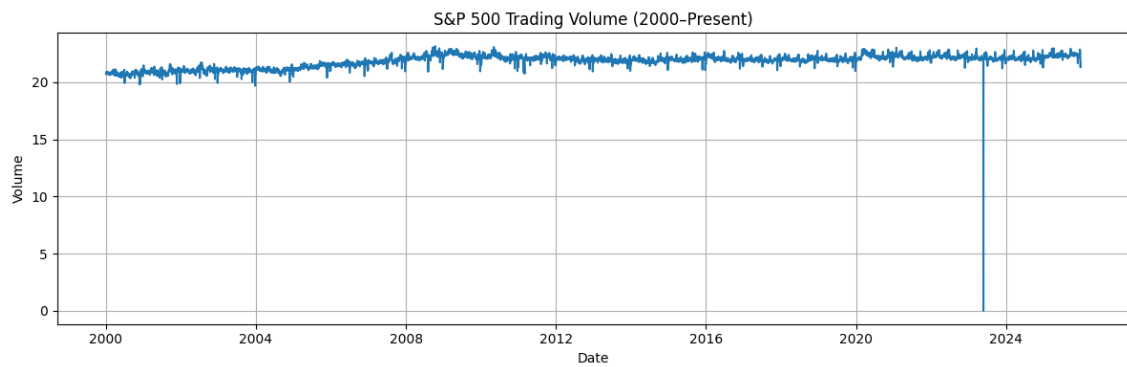
نمودارها

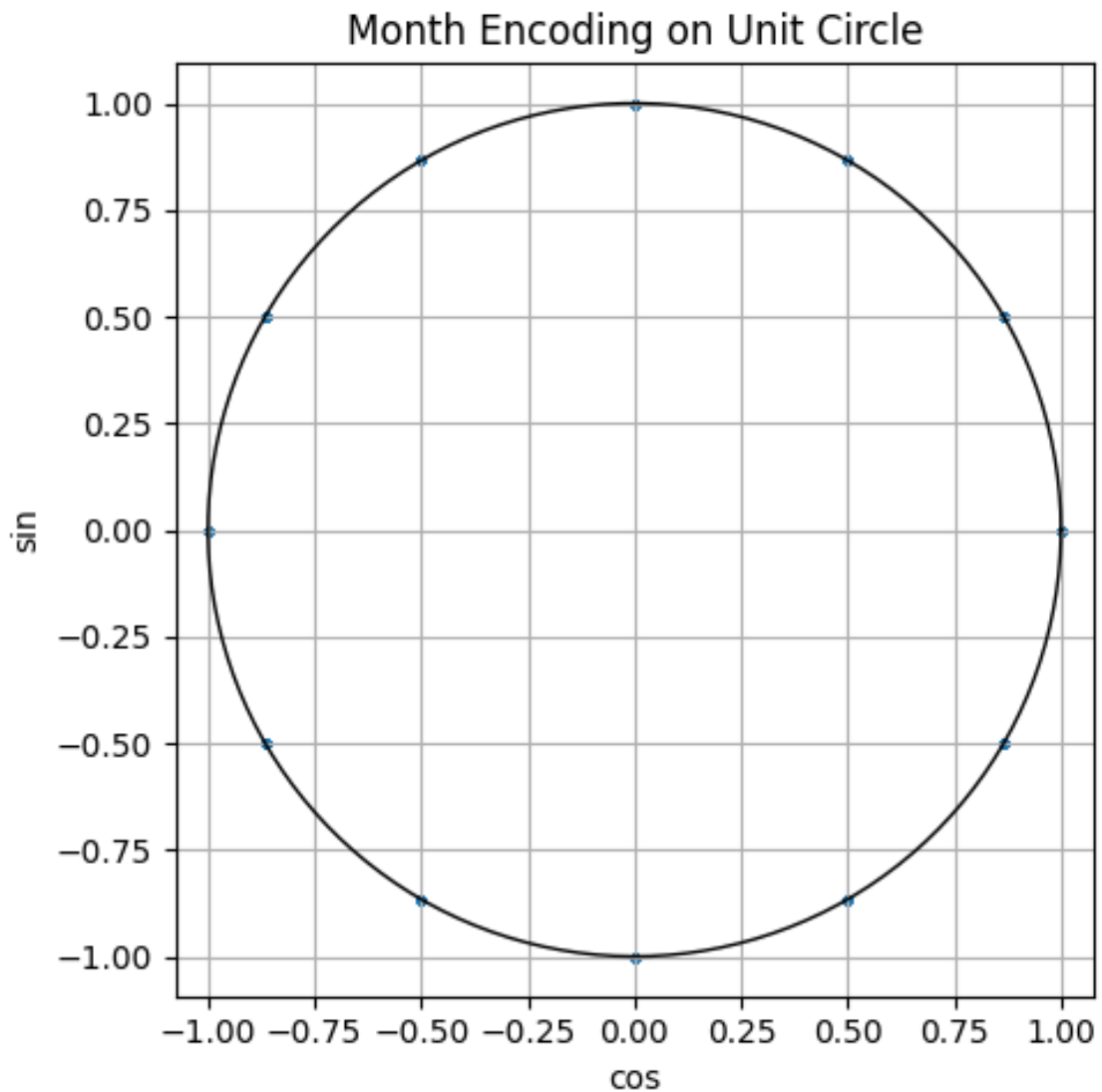


۲ پیش‌بینی قیمت سهام با استفاده از مدل‌های Recursive



۲ پیش‌بینی قیمت سهام با استفاده از مدل‌های Recursive





چرا از نرمال‌سازی Minmax استفاده می‌کنیم؟

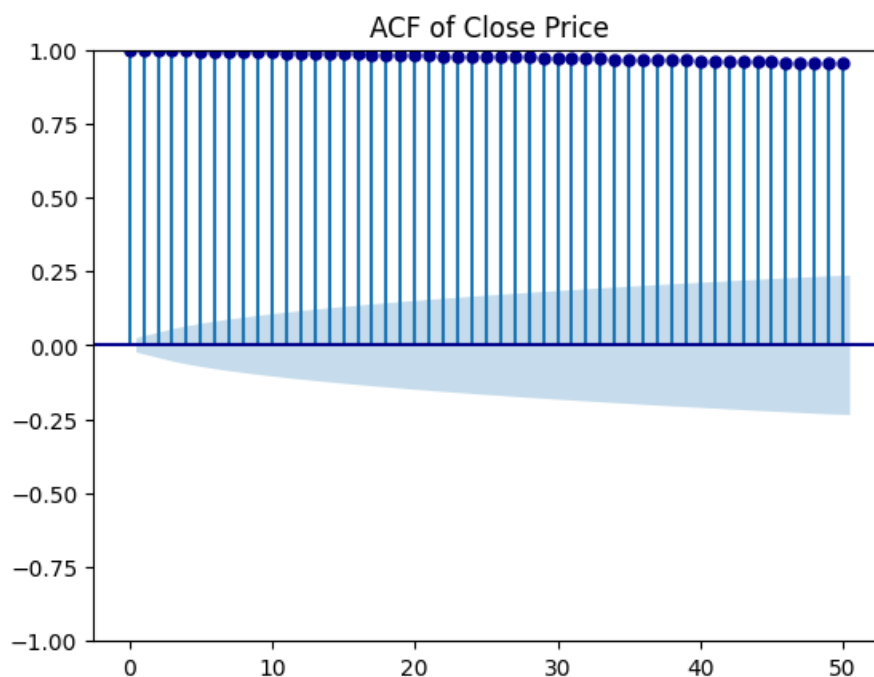
سال یک ویژگی زمانی خطی و غیرچرخه‌ای است که روند بلندمدت را نشان می‌دهد. نرمال‌سازی آن با Min-Max ترتیب زمانی را حفظ می‌کند و بدون القای تناوب مصنوعی، به بهبود آموزش مدل‌های یادگیری عمیق کمک می‌کند.

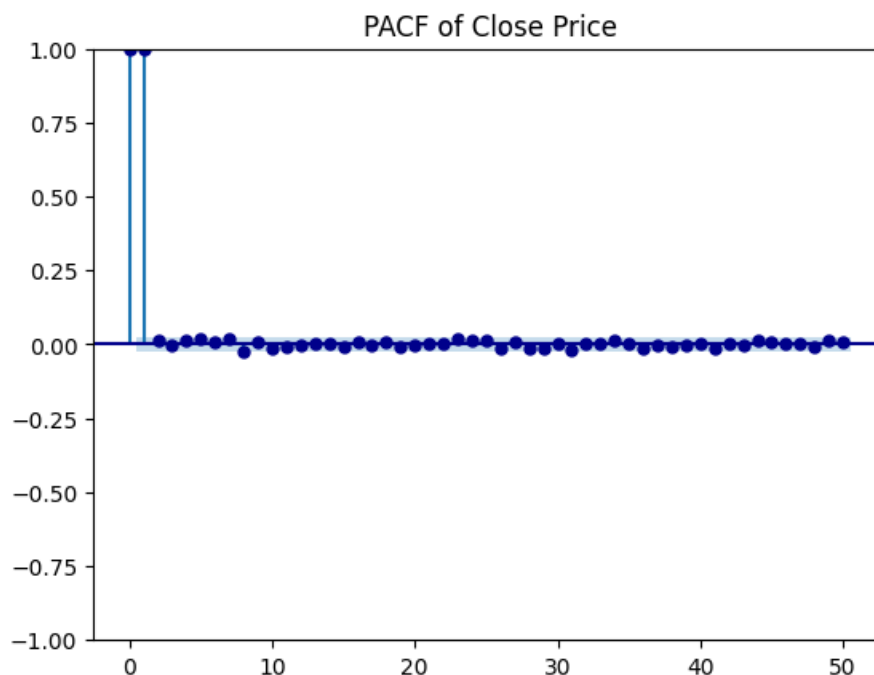
۲-۱-۲

ایستا کردن سیگنال

ایستا کردن سیگنال در تحلیل سری‌های زمانی به فرآیندی گفته می‌شود که طی آن ویژگی‌های آماری اصلی سیگنال، به‌ویژه ****میانگین و واریانس****، در طول زمان ثابت می‌شوند. بسیاری از سیگنال‌های واقعی، به‌خصوص سیگنال‌های مالی، دارای روند، تغییر واریانس یا رفتار تجمعی هستند که آن‌ها را غیر ایستا می‌کند. استفاده از این سیگنال‌های غیر ایستا در مدل‌سازی می‌تواند منجر به نتایج گمراه‌کننده و رگرسیون کاذب شود. برای ایستا کردن سیگنال معمولاً از روش‌هایی مانند ****تفاضل‌گیری****، حذف روند، یا تبدیل‌هایی نظیر لگاریتم استفاده می‌شود. با ایستا شدن سیگنال، روابط آماری مشاهده‌شده در گذشته معتبر باقی می‌مانند و امکان مدل‌سازی و پیش‌بینی قابل اعتماد آینده فراهم می‌شود.

نمودارهای ACF و PACF





نتایج ACF: مقادیر تابع خودهمبستگی (ACF) در تمامی لگ‌ها از ۱ تا ۱۰۰ معنادار گزارش شده‌اند. این رفتار نشان‌دهنده‌ی وابستگی شدید و بلندمدت سری زمانی به مقادیر گذشته است و بیان می‌کند که همبستگی‌ها به‌کندی کاهش می‌یابند. چنین الگویی مشخصه‌ی سری‌های غیرایستا و رونددار یا دارای رفتار شبه‌تصادفی (Random Walk) می‌باشد.

نتایج PACF: در نمودار خودهمبستگی جزئی (PACF) تنها لگ‌های ۱، ۸ و ۵۵ به‌عنوان لگ‌های معنادار شناسایی شده‌اند. با این حال، به دلیل غیرایستا بودن سری زمانی اولیه، تفسیر مستقیم این لگ‌ها برای تعیین مرتبه مدل قابل اتکا نیست و ابتدا لازم است سری زمانی ایستا شود.

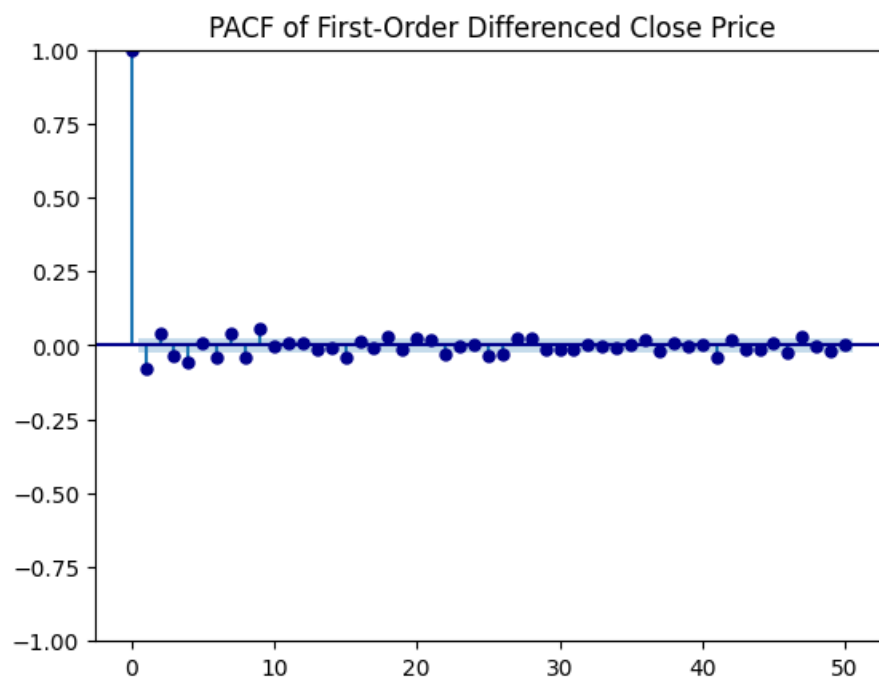
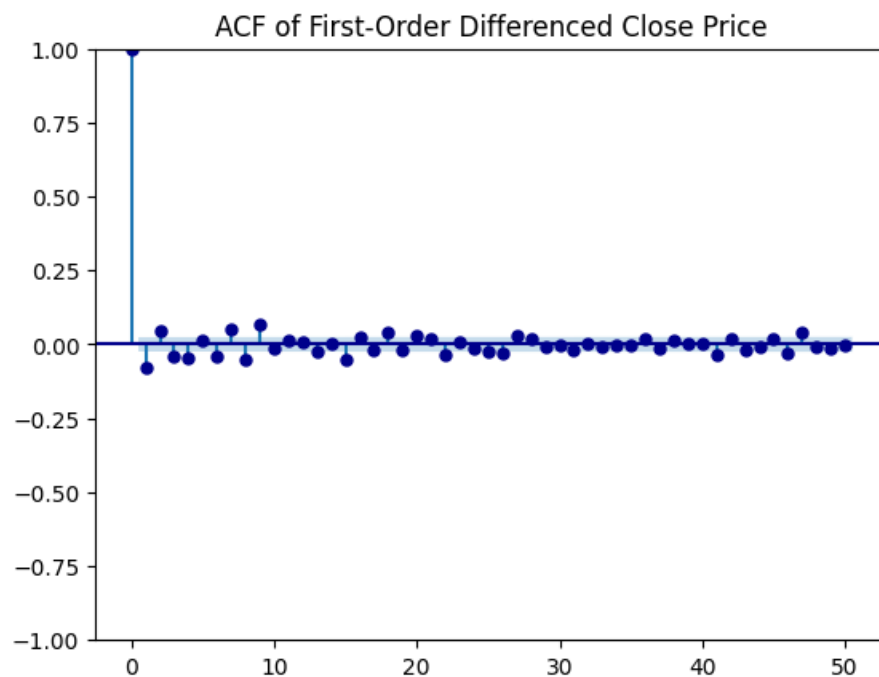
نتیجه‌گیری برای سری اولیه: رفتار توأمان ACF و PACF به‌وضوح نشان می‌دهد که سری زمانی اولیه ایستا نیست. استفاده مستقیم از این سری در مدل‌سازی می‌تواند منجر به رگرسیون کاذب شود؛ بنابراین، انجام تفاضل‌گیری به‌منظور ایستاسازی داده‌ها ضروری است.

چگونه می‌توان طول پنجره ورودی و افق پیش‌بینی را تعیین کرد؟

با توجه به تعداد lag ها با مقدار قابل توجه، به دو پنجره ورودی و خروجی تقسیم بندی بکنید و در نظر بگیرید که برای پنجره ورودی، سایز پنجره حداکثر برابر با تعداد تفاضل برای ایستا کردن سیگنال ضرب شده در تعداد lag های با مقدار قابل توجه در حالت ایستایی خواهد بود و برای خروجی، حداکثر به تعداد lag های قابل توجه در

حالت ایستا می‌توان طول پنجره را در نظر گرفت.

تحلیل نتایج و توضیح تغییرات حاصل از تفاضل‌گیری



نتایج ACF: پس از اعمال تفاضل مرتبه اول، تنها تعداد محدودی از لگ‌ها (مانند لگ‌های ۱ تا ۴، ۶ تا ۹، ۱۵، ۱۶، ۱۸ و چند لگ پراکنده دیگر) معنادار باقی مانده‌اند. این الگو بیانگر کاهش سریع خودهمبستگی و حذف وابستگی‌های بلندمدت است که از ویژگی‌های اصلی یک سری زمانی ایستا محسوب می‌شود.

نتایج PACF: در سری تفاضل‌گیری‌شده، لگ‌های ابتدایی (به‌ویژه لگ‌های ۱ تا ۴) بیشترین اهمیت را دارند و سایر لگ‌های معنادار به‌صورت پراکنده و در فواصل زمانی بزرگتر مشاهده می‌شوند. این موضوع نشان می‌دهد که وابستگی اصلی سری زمانی به لگ‌های کوتا‌مدت محدود شده است.

نتیجه‌گیری برای سری تفاضل‌گیری‌شده: پس از یک مرتبه تفاضل‌گیری، ساختار وابستگی سری زمانی به‌طور قابل‌توجهی ساده‌تر شده و سری به حالت ایستا رسیده است. در این شرایط، استفاده از لگ‌های کوتا‌مدت (به‌ویژه لگ‌های ۱ تا ۴) برای مدل‌سازی مناسب‌تر بوده و لگ‌های بلندمدت نقش ثانویه دارند.

تفسیر نتایج آزمون دیکی-فولر تعمیم‌یافته (ADF)

آزمون دیکی-فولر تعمیم‌یافته (ADF) برای بررسی وجود ریشه واحد در سری زمانی به‌کار می‌رود. فرضیه صفر این آزمون بیان می‌کند که سری زمانی غیر ایستا است.

* اگر مقدار $p\text{-value} < 0.05$ باشد، فرضیه صفر رد شده و سری زمانی ایستا است. * اگر مقدار $p\text{-value} > 0.05$ باشد، فرضیه صفر رد نمی‌شود و سری زمانی غیر ایستا باقی می‌ماند.

نتایج آزمون:

* سری قیمت پایانی اولیه: مقدار $p\text{-value}$ برابر با ۱ گزارش شده است که به‌طور قابل‌توجهی بزرگتر از ۰,۰۵ می‌باشد. این نتیجه نشان می‌دهد که سری زمانی اصلی غیر ایستا بوده و دارای رفتار رونددار یا شبه‌تصادفی است.

* سری تفاضل‌گیری‌شده (تفاضل مرتبه اول): مقدار $p\text{-value}$ برابر با ۰ گزارش شده است. این مقدار بسیار کوچک نشان‌دهنده رد فرضیه صفر و ایستا بودن سری زمانی پس از تفاضل‌گیری است.

۳-۱-۲

پروژه ارگودیک

یک پروژه تصادفی ارگودیک به پروژه‌ای گفته می‌شود که در آن ویژگی‌های آماری به‌دست‌آمده از یک مشاهده طولانی‌مدت در طول زمان (Time Average) با ویژگی‌های آماری کل فضای احتمالاتی آن پروژه (Ensemble Average) برابر باشد. به بیان دیگر، در یک پروژه ارگودیک، یک نمونه زمانی به‌اندازه کافی طولانی، نماینده‌ی

کل رفتار آماری سیستم محسوب می‌شود. در تحلیل سری‌های زمانی، ارگودیک بودن به این معناست که اطلاعات استخراج‌شده از یک دنباله زمانی منفرد، برای توصیف کامل ساختار آماری فرآیند کفایت می‌کند. لازم به ذکر است که ایستایی شرط لازم برای ارگودیک بودن است؛ بنابراین، یک سری زمانی غیر ایستا نمی‌تواند ارگودیک باشد.

ارگودیک بودن سیگنال‌ها برای مدل‌های یادگیری عمیق

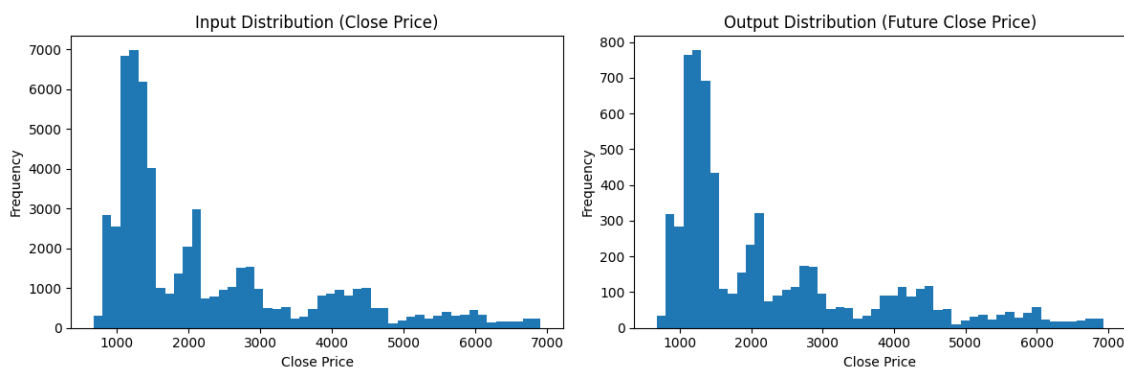
ارگودیک بودن داده‌ها نقش مهمی در موفقیت مدل‌های یادگیری عمیق مانند شبکه‌های LSTM یا Trans-former ایفا می‌کند. این مدل‌ها بر این فرض استوار هستند که توزیع آماری داده‌های آموزش و آزمون یکسان یا بسیار مشابه است. در صورتی که سیگنال ارگودیک باشد، الگوهای آماری مشاهده‌شده در داده‌های گذشته نماینده‌ی واقعی رفتار سیستم بوده و قابلیت تعمیم به آینده را دارند. در مقابل، اگر فرآیند غیر ارگودیک باشد و قوانین آماری آن در طول زمان تغییر کند، مدل تنها بر گذشته منطبق شده و در پیش‌بینی داده‌های آینده با شکست مواجه می‌شود.

رابطه ایستاسازی و ارگودیک بودن

در عمل، هدف اصلی در پیش‌پردازش داده‌های سری زمانی، ایستا کردن سیگنال است، نه ارگودیک کردن مستقیم آن. دلیل این موضوع آن است که ابزار مستقیمی برای تحمیل ارگودیک بودن وجود ندارد، اما از نظر تئوریک می‌دانیم که ایستایی شرط لازم برای ارگودیک بودن محسوب می‌شود. با استفاده از روش‌هایی مانند تفاضل‌گیری یا حذف روند، سیگنال به حالتی تبدیل می‌شود که میانگین و واریانس آن در طول زمان ثابت باشد. در بسیاری از کاربردهای عملی، پس از ایستا شدن سری زمانی، فرض می‌شود که سیگنال از نظر عملی رفتار ارگودیک دارد و می‌توان از آن برای مدل‌سازی استفاده کرد.

ارگودیک بودن قیمت‌های open و close

قیمت‌های خام مالی مانند Open یا Close معمولاً غیر ایستا هستند و دارای روند یا رفتار تجمعی می‌باشند؛ در نتیجه، این سیگنال‌ها ارگودیک نیستند و استفاده مستقیم از آن‌ها در مدل‌سازی توصیه نمی‌شود. در مقابل، زمانی که از قیمت‌ها تفاضل‌گیری انجام می‌شود یا بازده قیمت‌ها مورد استفاده قرار می‌گیرد، سیگنال حاصل معمولاً حول مقدار ثابتی (نزدیک به صفر) نوسان کرده و ایستا می‌شود. این نوع سیگنال‌ها به‌طور تقریبی دارای ویژگی‌های ارگودیک بوده و برای استفاده در مدل‌های یادگیری ماشین و یادگیری عمیق مناسب‌تر هستند.



انتخاب اندازه پنجره ورودی و خروجی

بر اساس تحلیل نمودارهای ACF و PACF در بخش‌های قبلی، مشخص شد که بیشترین وابستگی معنادار سری زمانی به لگ‌های کوتاه‌مدت و به‌ویژه تا حدود لگ ۹ محدود می‌شود. از این‌رو، اندازه پنجره ورودی Input Window Size برابر با ۹ انتخاب شد تا اطلاعات زمانی مؤثر گذشته به‌طور کامل در اختیار مدل قرار گیرد. همچنین، با توجه به هدف پیش‌بینی مقدار گام بعدی در سری زمانی، اندازه پنجره خروجی Output Window Size برابر با ۱ در نظر گرفته شد که متناسب با ساختار مسئله و تحلیل انجام‌شده است.

مفهوم نشت داده (Data Leakage)

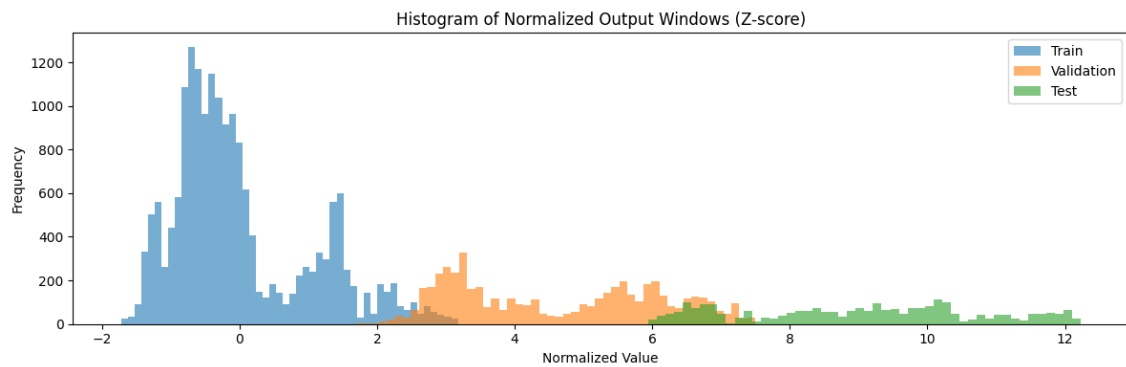
نشت داده به حالتی گفته می‌شود که اطلاعات مربوط به داده‌های آزمون یا آینده به‌صورت مستقیم یا غیرمستقیم در فرآیند آموزش مدل مورد استفاده قرار گیرد. این اتفاق باعث می‌شود عملکرد مدل به‌صورت غیرواقعی بالا ارزیابی شود، در حالی که مدل در شرایط واقعی توان تعمیم مناسبی نخواهد داشت. نشت داده یکی از خطاهای رایج و بسیار مهم در مدل‌سازی سری‌های زمانی و یادگیری ماشین محسوب می‌شود.

در این پژوهش، برای جلوگیری از نشت داده، فرآیند نرمال‌سازی ویژگی‌ها به‌صورت کاملاً صحیح انجام شده است. همان‌طور که در کد مشاهده می‌شود، پارامترهای مقیاس‌بندی برای داده‌های ورودی (X) و خروجی (Y) تنها بر اساس داده‌های آموزشی (Training Data) برآورد شده‌اند. سپس همان پارامترها بدون هیچ‌گونه بازآموزی، برای تبدیل داده‌های اعتبارسنجی (Validation) و آزمون (Test) و سال ۲۰۲۴ مورد استفاده قرار گرفته‌اند.

این رویکرد تضمین می‌کند که هیچ‌گونه اطلاعات آماری از داده‌های آینده وارد فرآیند آموزش نشده و مدل صرفاً بر اساس اطلاعات گذشته آموزش دیده است. در نتیجه، نشت داده در فرآیند آماده‌سازی داده‌ها رخ نداده است و نتایج حاصل از مدل‌سازی قابل اعتماد و قابل تعمیم می‌باشند.

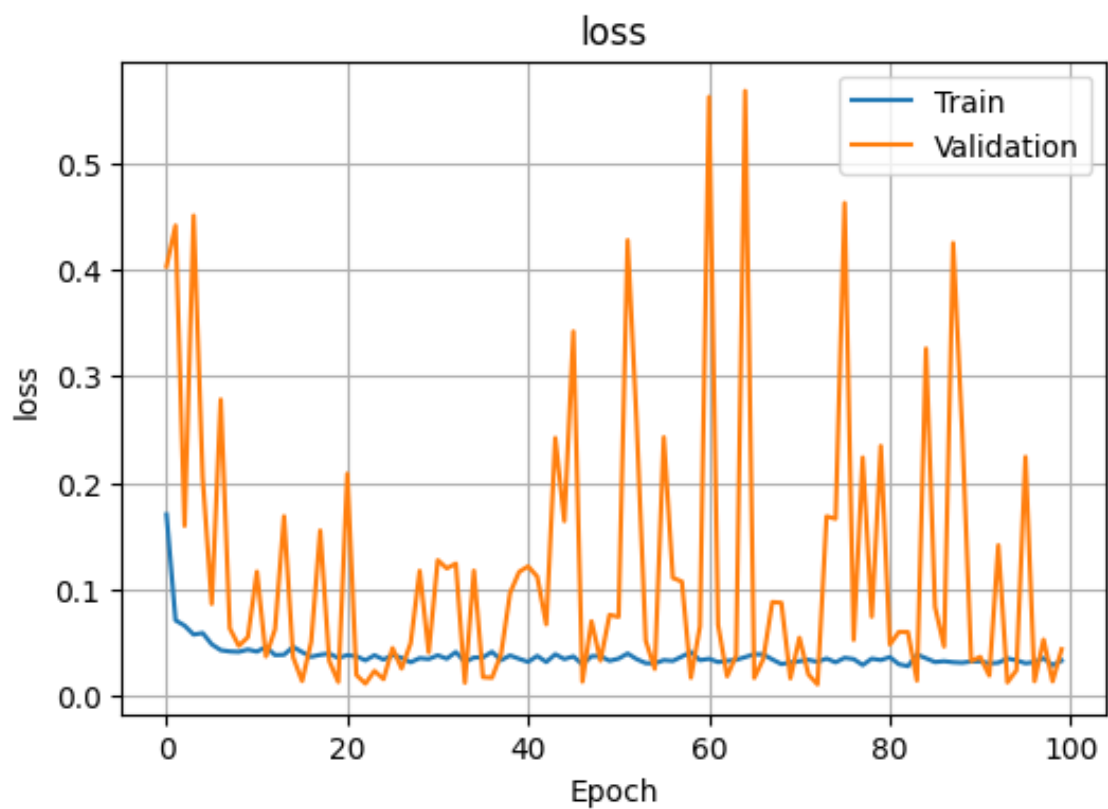
هیستوگرام پنجره‌های خروجی

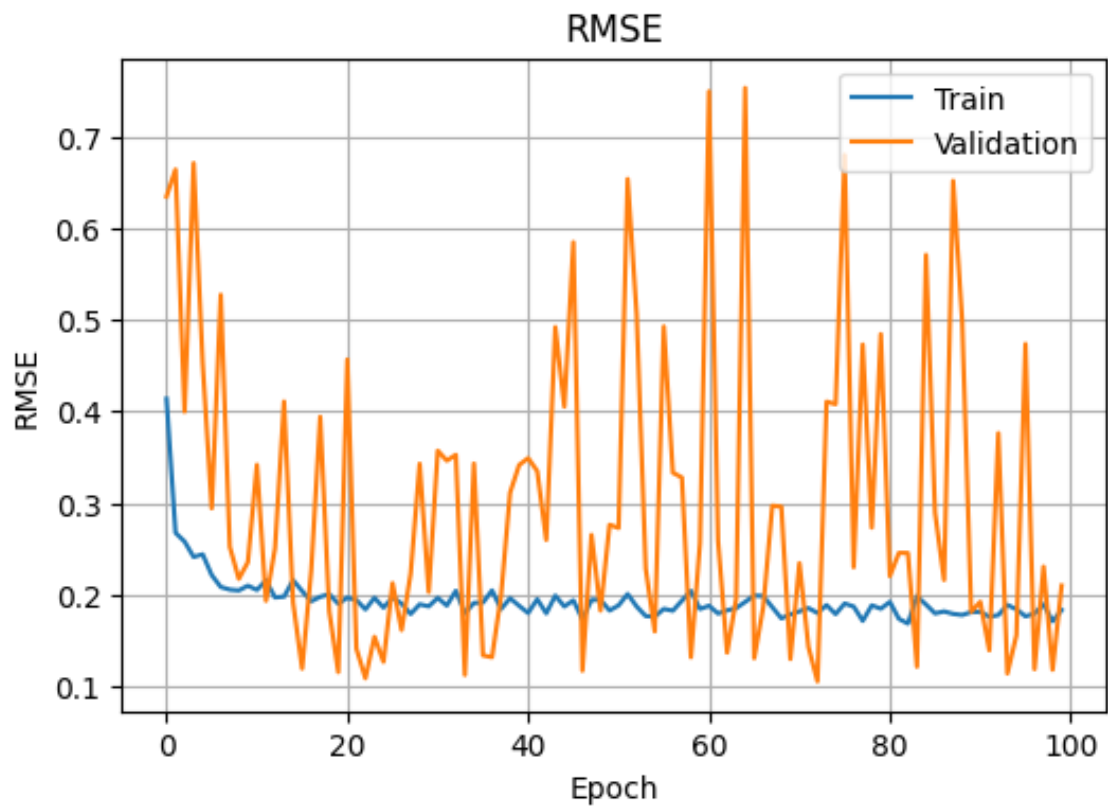
۲ پیش‌بینی قیمت سهام با استفاده از مدل‌های Recursive

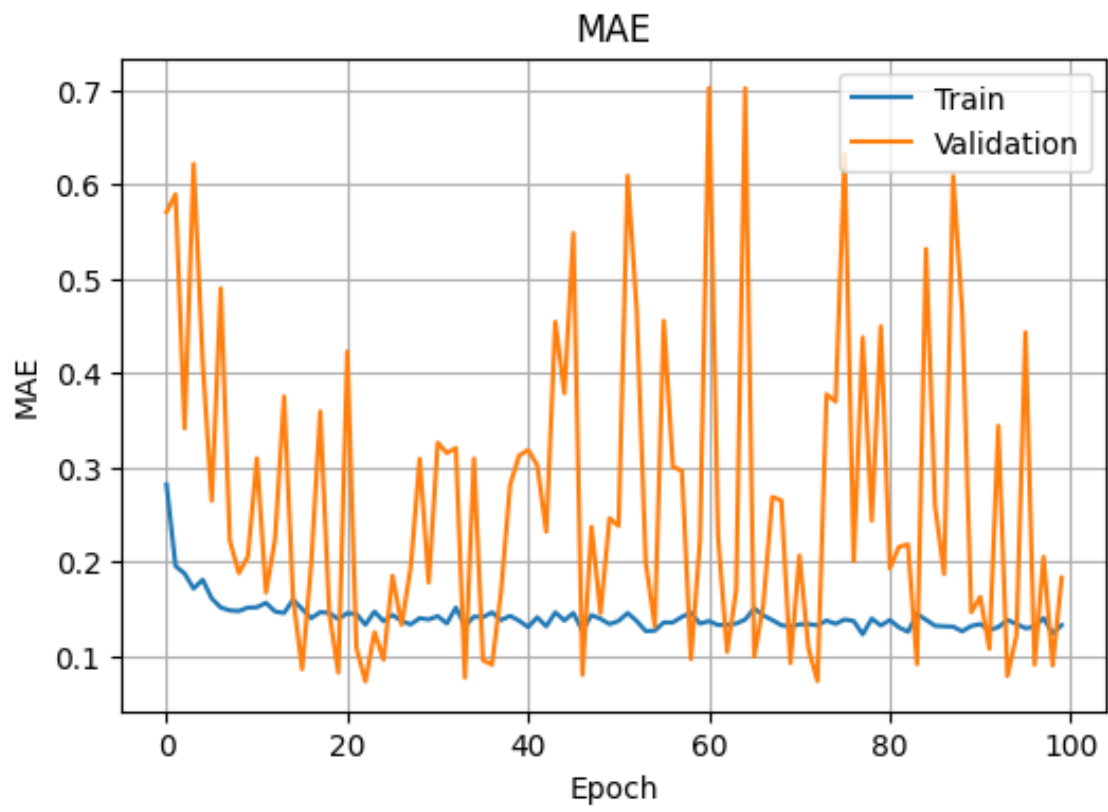


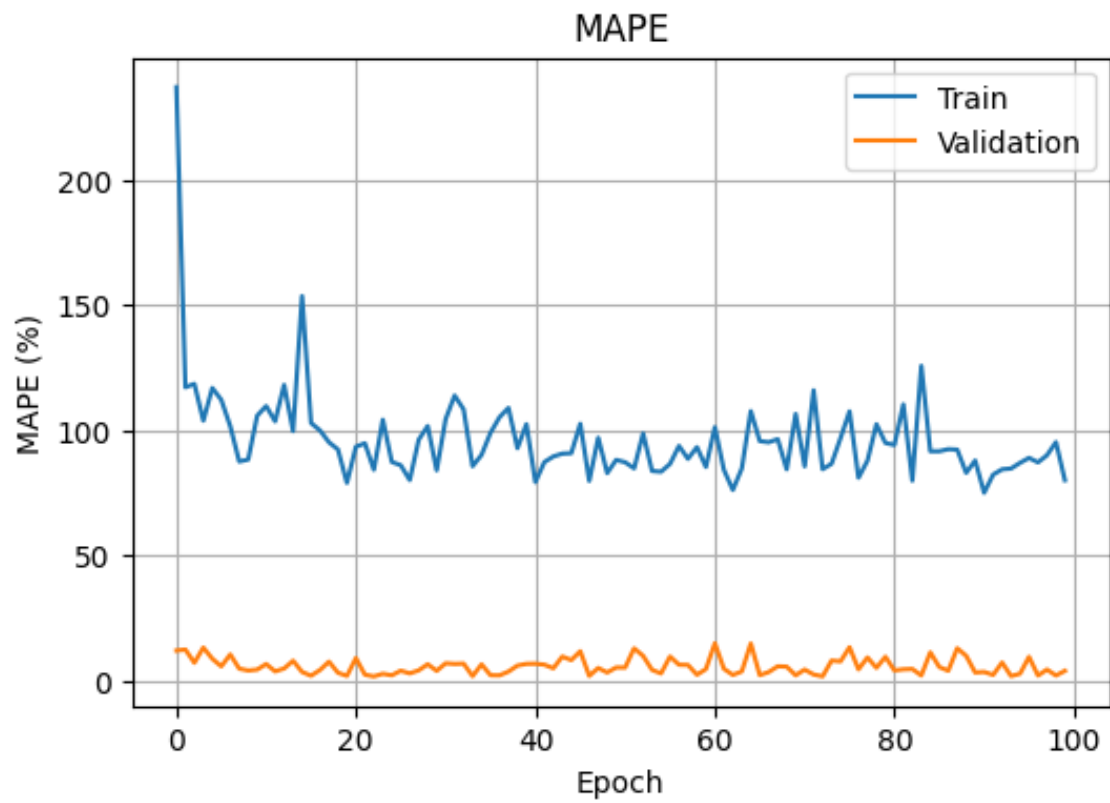
۲-۲ مدل پایه

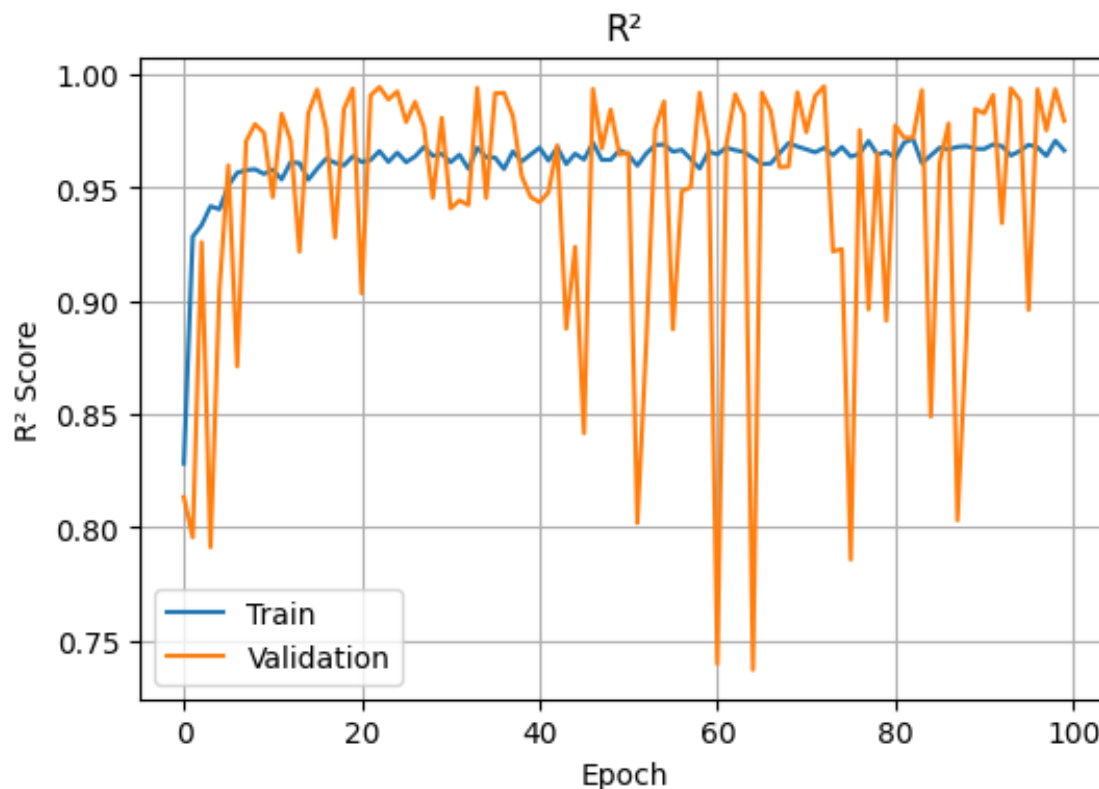
۱-۲-۲ نمودارها











تحلیل نمودارهای ارزیابی مدل

• نمودار تابع هزینه (Loss)

نمودار خطای آموزش (Train) رفتار بسیار هموار و کاهشی را نشان می‌دهد و در مدت‌زمان کوتاهی به مقادیری نزدیک به صفر همگرا می‌شود. این موضوع بیانگر آن است که مدل توانسته الگوی موجود در داده‌های آموزشی را به‌خوبی یاد بگیرد. در مقابل، نمودار خطای اعتبارسنجی (Validation) دارای نوسانات شدید و جهش‌های ناگهانی (Spikes) است. هرچند در برخی دوره‌ها مقدار خطا کاهش می‌یابد، اما افزایش‌های ناگهانی و مکرر نشان می‌دهد که عملکرد مدل روی داده‌های جدید پایدار نیست.

• نمودارهای RMSE و MAE

رفتار نمودارهای RMSE و MAE کاملاً مشابه نمودار Loss است. مقادیر خطای آموزش پایین و پایدار بوده، در حالی که خطای اعتبارسنجی دارای نوسانات قابل‌توجهی است. این الگو نشان می‌دهد که مدل در

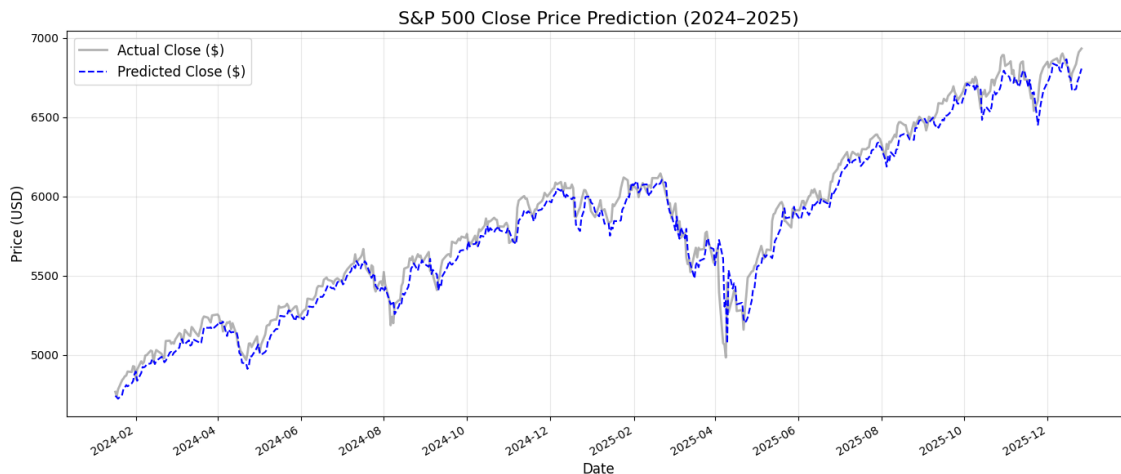
مواجهه با داده‌های دیدن‌نشده، در برخی دوره‌های آموزشی دچار خطاهای بزرگ می‌شود و قابلیت تعمیم آن محدود است.

- نمودار MAPE (میانگین درصد خطای مطلق)

در این نمودار، رفتاری غیرمنتظره مشاهده می‌شود؛ به‌طوری‌که مقدار MAPE در مجموعه آموزشی بسیار بالا (حدود ۱۰۰٪) و در مجموعه اعتبارسنجی بسیار پایین (نزدیک به صفر) است. این تضاد با سایر معیارها معمولاً ناشی از وجود مقادیر بسیار کوچک یا نزدیک به صفر در متغیر هدف مجموعه آموزشی است. از آنجا که MAPE شامل تقسیم بر مقدار واقعی هدف می‌باشد، وجود چنین مقادیری می‌تواند باعث انفجار مقدار خطا شود. در مقابل، به نظر می‌رسد که مجموعه اعتبارسنجی فاقد این مقادیر خاص بوده است. این مسئله نشان‌دهنده لزوم بازبینی پیش‌پردازش داده‌ها و مناسب‌بودن معیار MAPE برای این مسئله است.

- نمودار ضریب تعیین (R^2 Score)

امتیاز ضریب تعیین (R^2) برای هر دو مجموعه آموزش و اعتبارسنجی مقادیر بالایی (بیش از ۹۰،۰) را نشان می‌دهد که بیانگر توان توضیح‌دهندگی بالای مدل است. در برخی دوره‌ها، عملکرد مجموعه اعتبارسنجی حتی بهتر از مجموعه آموزشی بوده است که این پدیده در مجموعه داده‌های محدود ممکن است رخ دهد. با این حال، افت‌های ناگهانی و شدید در نمودار R^2 مربوط به مجموعه اعتبارسنجی مشاهده می‌شود که بیانگر عدم پایداری عملکرد مدل در طول فرآیند آموزش است.



تحلیل نمودار

این نمودار مقایسه‌ای بین قیمت واقعی شاخص SP500 و مقادیر پیش‌بینی‌شده توسط مدل سری زمانی را در بازه زمانی حدود دو ساله نشان می‌دهد. نتایج حاکی از همبستگی بسیار بالا بین مقادیر واقعی و پیش‌بینی‌شده است که بیانگر یادگیری مناسب الگوهای اصلی بازار توسط مدل می‌باشد. مدل توانسته است روندهای صعودی و نزولی بازار را به‌خوبی شناسایی کرده و حتی در دوره‌های نوسان شدید، مانند افت شدید بازار در حوالی آوریل ۲۰۲۵، رفتار کلی قیمت را به‌درستی دنبال کند.

با وجود این عملکرد مطلوب، یک کم‌رآوردی سیستماتیک در پیش‌بینی‌ها مشاهده می‌شود؛ به‌طوری‌که در اغلب نقاط، مقادیر پیش‌بینی‌شده اندکی پایین‌تر از مقادیر واقعی قرار دارند. همچنین، در نقاط چرخش روند، مقدار پیش‌بینی با تأخیر زمانی نسبت به مقدار واقعی تغییر جهت می‌دهد که نشان‌دهنده وابستگی مدل به مقادیر گذشته است. به‌طور کلی، با توجه به نزدیکی دو منحنی، انتظار می‌رود معیارهایی مانند R^2 مقدار بالایی داشته باشند، هرچند اصلاح بایاس منفی می‌تواند دقت مدل را برای کاربردهای عملی و معاملاتی افزایش دهد.

معماری شبکه و پارامترها

Layer (type:depth-idx)	Output Shape	Param
BaselineMLP	[1, 4]	--

۲ پیش‌بینی قیمت سهام با استفاده از مدل‌های Recursive

Sequential: 11-	[1, 4]	--
Flatten: 21-	[1, 90]	--
Linear: 22-	[1, 128]	11,648
BatchNorm1d: 23-	[1, 128]	256
ReLU: 24-	[1, 128]	--
Dropout: 25-	[1, 128]	--
Linear: 26-	[1, 64]	8,256
BatchNorm1d: 27-	[1, 64]	128
ReLU: 28-	[1, 64]	--
Dropout: 29-	[1, 64]	--
Linear: 210-	[1, 4]	260

=====
Total params: 20,548

Trainable params: 20,548

Non-trainable params: 0

Total mult-adds (Units.MEGABYTES): 02.0
=====

Input size (MB): 00.0

Forward/backward pass size (MB): 00.0

Params size (MB): 08.0

Estimated Total Size (MB): 09.0
=====

۳-۲ مدل‌های Recursive Neural Network

۱-۳-۲ معماری LSTM

۱.۱.۳.۲

تحلیل معماری CNN-LSTM با اتصال باقی‌مانده (Residual)

کد ارائه‌شده یک پیاده‌سازی صحیح و استاندارد از معماری CNN-LSTM با رویکرد باقی‌مانده (Residual) برای پیش‌بینی سری‌های زمانی مالی است. در این بخش، نحوه عملکرد اجزای اصلی مدل و جریان اطلاعات در شبکه تشریح می‌شود.

نقش و عملکرد لایه‌های کانولوشنی (CNN)

ورودی لایه‌های کانولوشنی شامل کل پنجره ورودی سری زمانی است؛ به عنوان مثال، چندین روز متوالی از داده‌های قیمتی (مانند Low High، Open، Close). لایه کانولوشن با استفاده از کرنل‌هایی با اندازه ثابت (در این مدل با اندازه ۳)، روی محور زمان حرکت کرده و روابط محلی بین چند گام زمانی متوالی را استخراج می‌کند. این لایه به جای تحلیل کل دنباله به صورت یکجا، به شناسایی الگوهای کوتاه مدت و محلی مانند روندهای چندروزه یا ساختارهای قیمتی ساده می‌پردازد. در نتیجه، نویزهای جزئی کاهش یافته و نگاشت‌هایی از ویژگی‌ها (Feature Maps) تولید می‌شود که نمایانگر رفتار فشرده و معنادار سری زمانی در بازه‌های کوتاه هستند.

انتقال ویژگی‌ها از CNN به LSTM

پس از اعمال لایه کانولوشن و تابع فعال‌سازی، خروجی CNN به صورت تانسوری با ابعاد (Batch, Chan, nels, Sequence Length) تولید می‌شود. برای سازگاری با ورودی LSTM، ابعاد این تانسور با استفاده از عملگر تغییر ترتیب محورها (permute) به شکل (Batch, Sequence Length, Features) تبدیل می‌شود. در این مرحله، LSTM به جای مشاهده داده‌های خام قیمتی، مجموعه‌ای از ویژگی‌های استخراج‌شده و غنی‌شده توسط CNN را دریافت می‌کند. بدین ترتیب، وظیفه LSTM مدل‌سازی وابستگی‌های زمانی بلندمدت میان این الگوهای محلی و کشف ساختار کلی سری زمانی است.

استفاده از آخرین وضعیت مخفی LSTM برای پیش‌بینی

برای تولید خروجی نهایی، از آخرین وضعیت مخفی (Last Hidden State) لایه LSTM استفاده شده است. این انتخاب از نظر فنی مناسب و متداول است، به ویژه در مسائل پیش‌بینی از نوع Many-to-One (چند گام ورودی و یک گام خروجی).

خروجی کامل LSTM شامل بردارهایی برای تمام گام‌های زمانی ورودی است که استفاده مستقیم از آن‌ها مستلزم افزایش شدید تعداد پارامترها و احتمال بیش‌برازش می‌باشد. در مقابل، آخرین وضعیت مخفی خلاصه‌ای فشرده از کل اطلاعات زمانی پنجره ورودی را در خود نگه می‌دارد و گزینه‌ای کارآمد برای تغذیه لایه کاملاً متصل (Fully Connected) محسوب می‌شود.

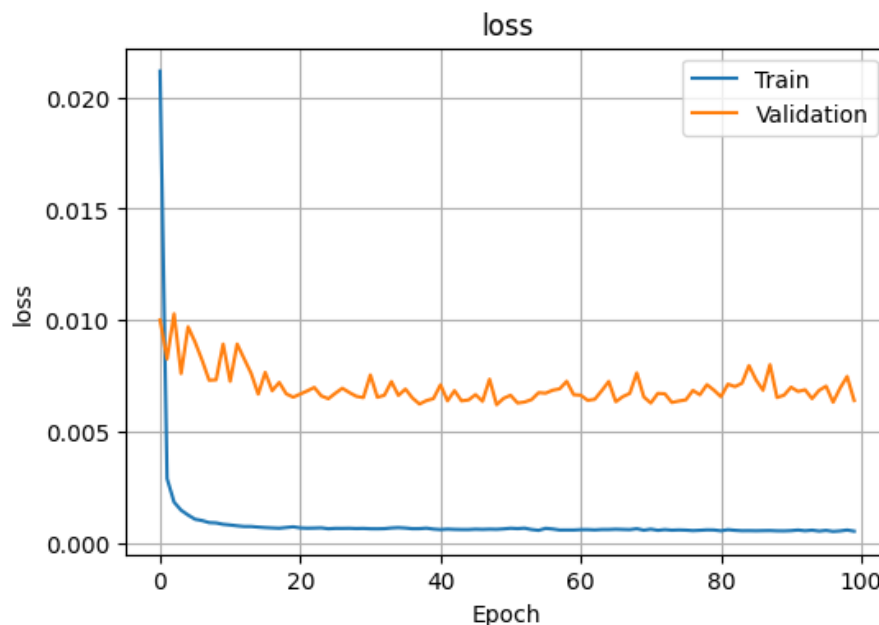
در کد پیاده‌سازی‌شده، استفاده از بردار $h_n[-1]$ که متناظر با آخرین وضعیت مخفی لایه نهایی LSTM است، انتخابی صحیح و همسو با هدف مدل‌سازی انجام‌شده می‌باشد.

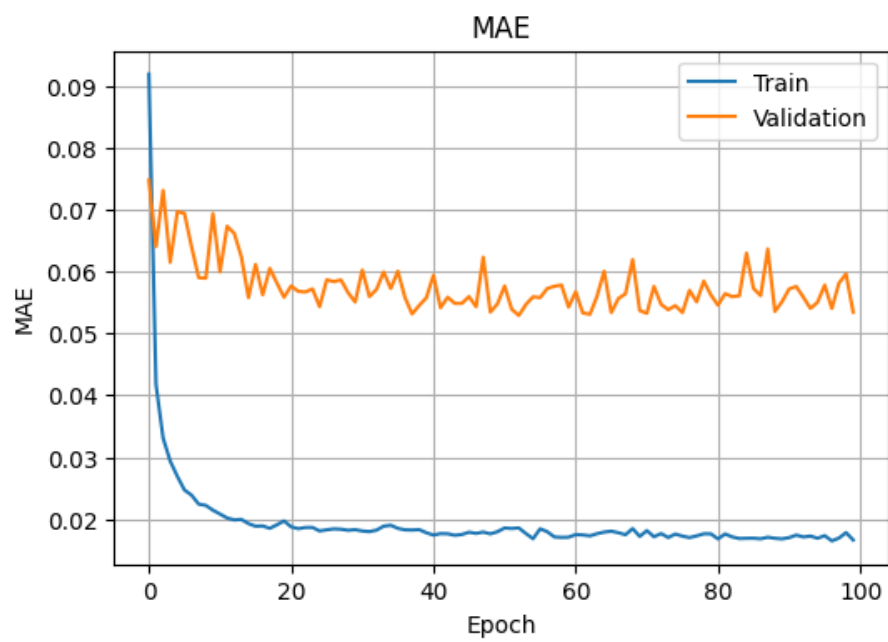
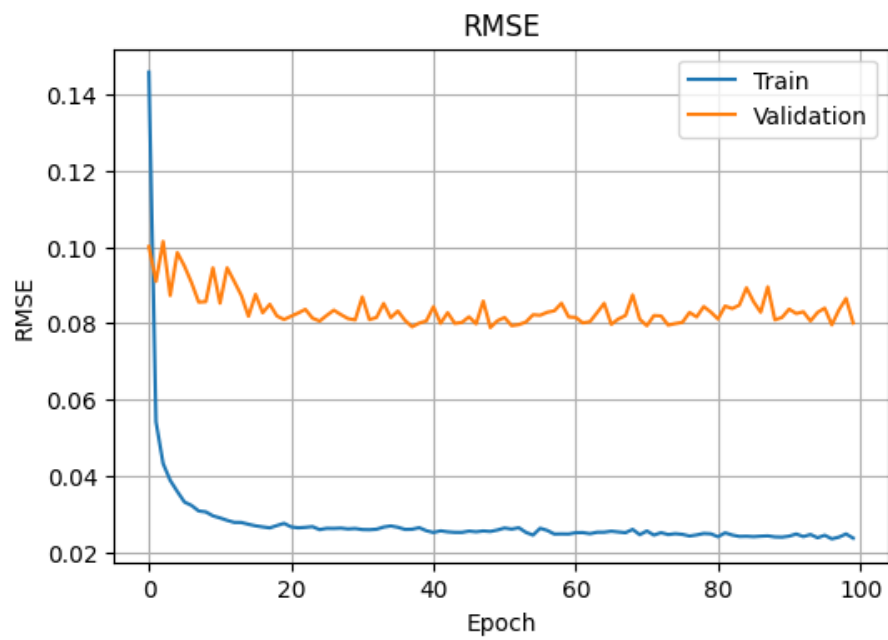
نقش اتصال باقی‌مانده (Residual Connection)

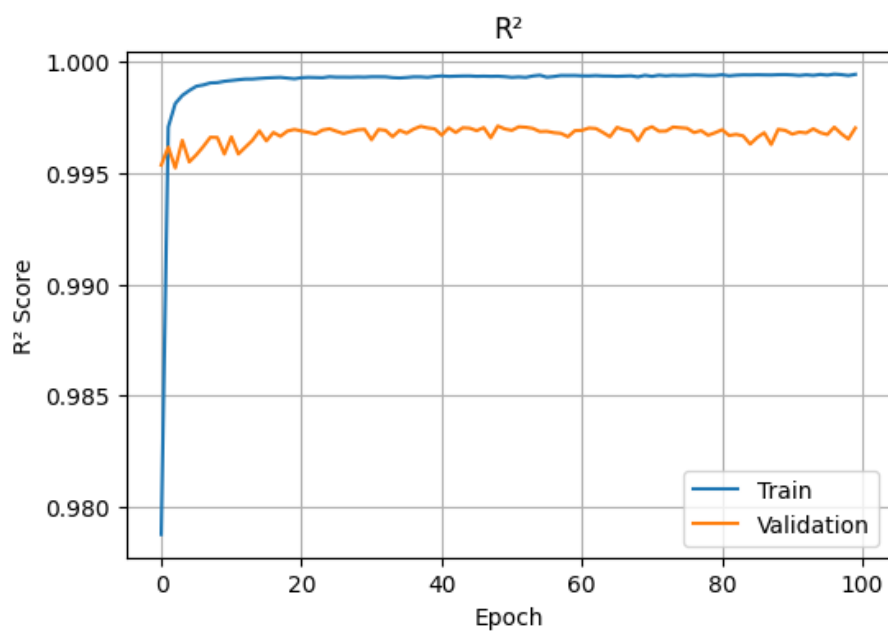
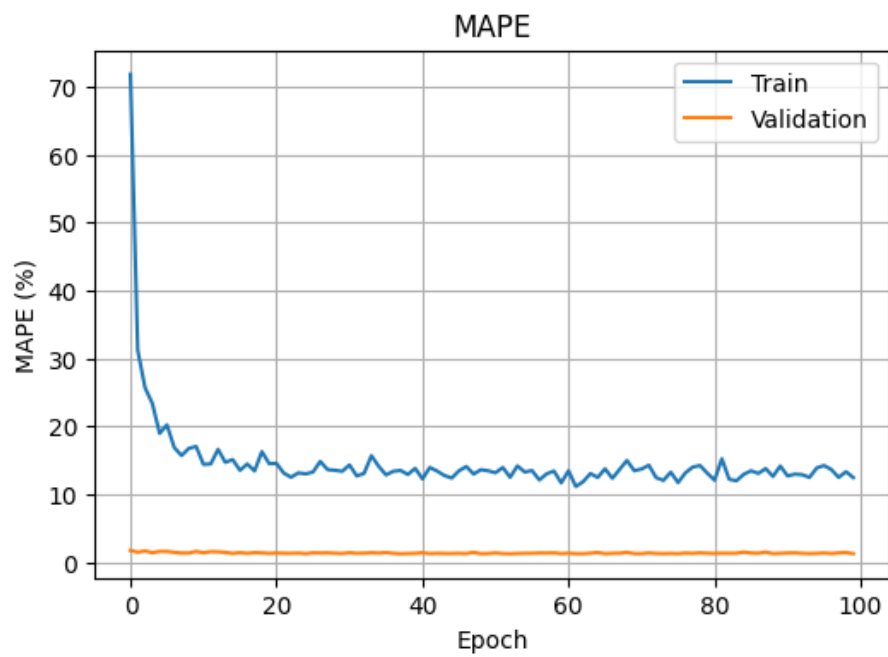
در معماری ارائه‌شده، خروجی مدل به‌صورت یک مقدار افزایشی (Delta) نسبت به آخرین قیمت مشاهده‌شده پیش‌بینی می‌شود و سپس با مقدار واقعی قیمت قبلی جمع می‌گردد. این رویکرد باقی‌مانده باعث می‌شود مدل به‌جای پیش‌بینی مستقیم قیمت مطلق، بر یادگیری تغییرات قیمت تمرکز کند که از نظر آماری ساده‌تر و پایدارتر است.

۲.۱.۳.۲

نمودارها







معماری و پارامترها

=====

Layer (type:depth-idx)

Output Shape

Param

CNN_LSTM	[1, 4]	--
Conv1d: 11-	[1, 64, 9]	1,984
BatchNorm1d: 12-	[1, 64, 9]	128
ReLU: 13-	[1, 64, 9]	--
LSTM: 14-	[1, 9, 64]	66,560
BatchNorm1d: 15-	[1, 64]	128
Linear: 16-	[1, 4]	260

Total params: 69,060

Trainable params: 69,060

Non-trainable params: 0

Total mult-adds (Units.MEGABYTES): 62.0

Input size (MB): 00.0

Forward/backward pass size (MB): 01.0

Params size (MB): 28.0

Estimated Total Size (MB): 29.0

آیا شبکه این قسمت بهتر از baseline عمل کرده است؟

بله، مدل CNN-LSTM عملکرد بهتری نسبت به MLP دارد. این مدل با داشتن خطای اعتبارسنجی کمتر، مقدار R^2 بالاتر و پایداری بیشتر در طول آموزش، توانسته است الگوهای زمانی داده را مؤثرتر یاد بگیرد. در مقابل، مدل MLP نوسانات قابل‌توجهی در خطای اعتبارسنجی نشان می‌دهد که بیانگر پایداری کمتر و ضعف در مدل‌سازی وابستگی‌های زمانی است.

۲-۳-۲ معماری GRU

۱.۲.۳.۲

ساختار شبکه

Layer (type:depth-idx)	Output Shape	Param
CNN_GRU	[1, 4]	--
Conv1d: 11-	[1, 64, 9]	1,984
BatchNorm1d: 12-	[1, 64, 9]	128
ReLU: 13-	[1, 64, 9]	--
GRU: 14-	[1, 9, 64]	49,920
BatchNorm1d: 15-	[1, 64]	128
Linear: 16-	[1, 4]	260
Total params: 52,420		
Trainable params: 52,420		
Non-trainable params: 0		
Total mult-adds (Units.MEGABYTES): 47.0		
Input size (MB): 00.0		
Forward/backward pass size (MB): 01.0		
Params size (MB): 21.0		
Estimated Total Size (MB): 22.0		

آیا استفاده از hidden feature ها در لایه آخر مناسب تر است و یا آخرین خروجی GRU؟

در ساختار GRU تنها یک حالت وجود دارد که همان Hidden State است. خروجی‌های GRU شامل دو بخش هستند:

- output که وضعیت مخفی تمام گام‌های زمانی را شامل می‌شود،

- h_n که آخرین وضعیت مخفی هر لایه را نگه می‌دارد.

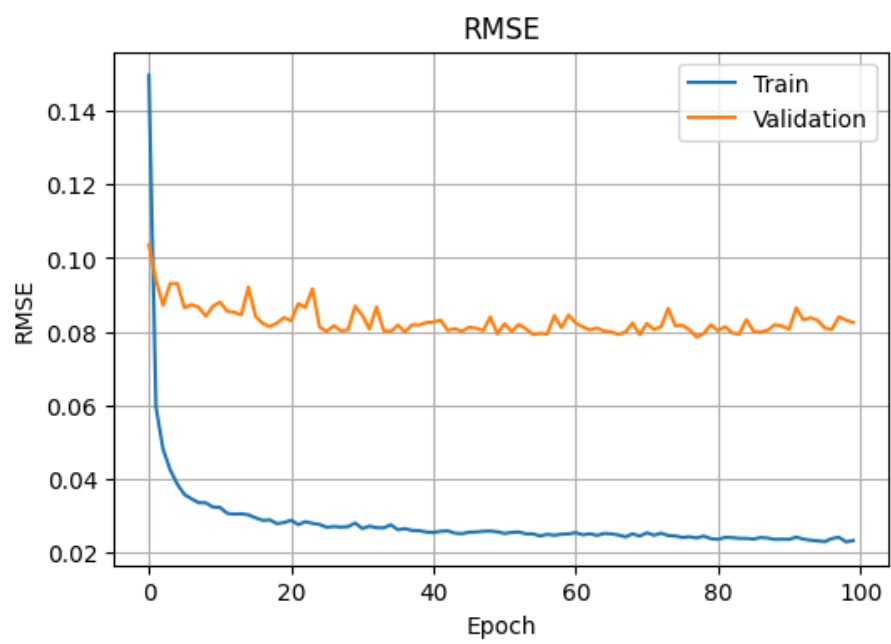
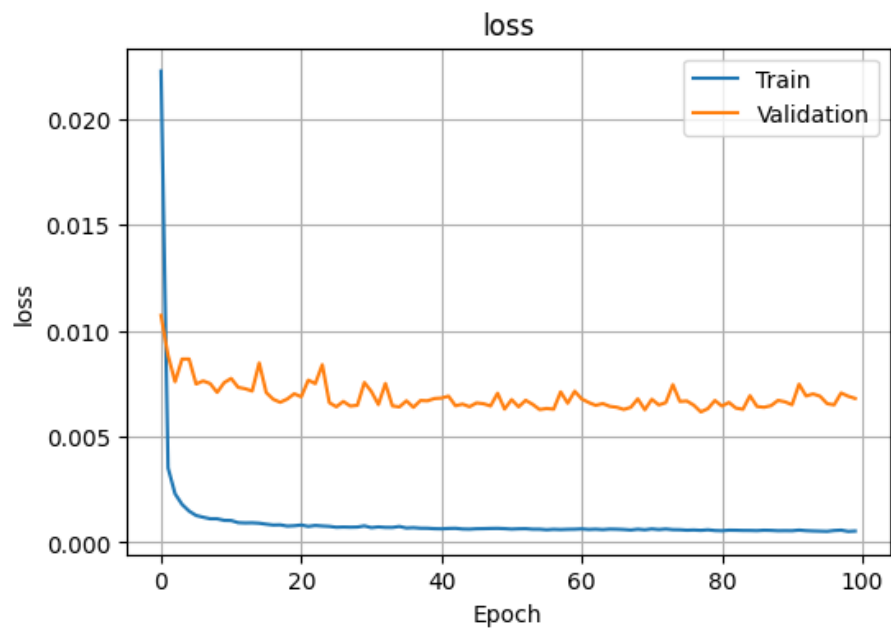
از آنجا که مسئله از نوع Many-to-One است (استفاده از یک پنجره زمانی برای پیش‌بینی یک مقدار آینده)، بردار $h_n[-1]$ که آخرین Hidden State لایه نهایی GRU است، به‌عنوان Context Vector عمل کرده و چکیده‌ای از کل اطلاعات توالی زمانی را در خود دارد. بنابراین این بردار بهترین ورودی برای لایه Fully Connected نهایی محسوب می‌شود.

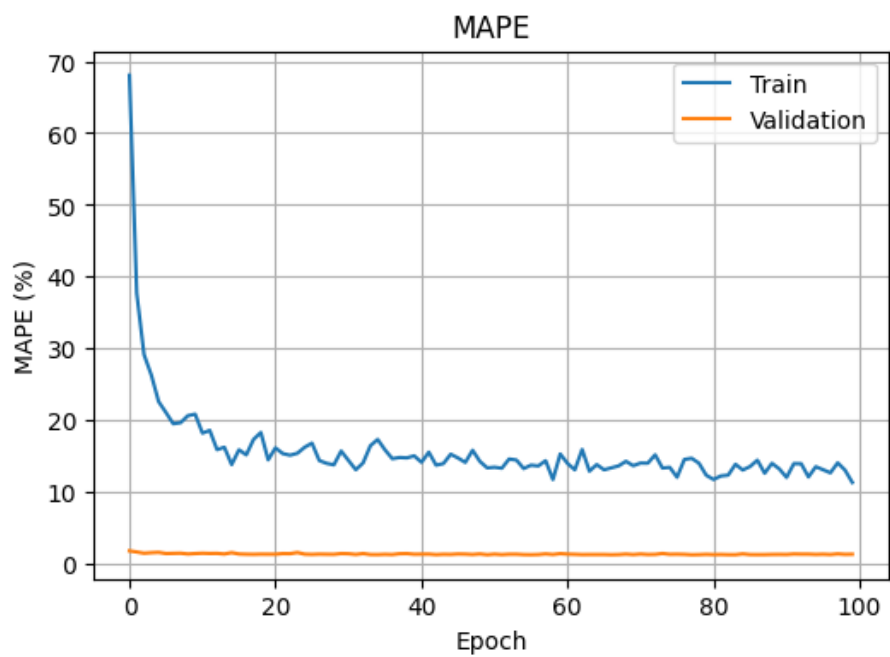
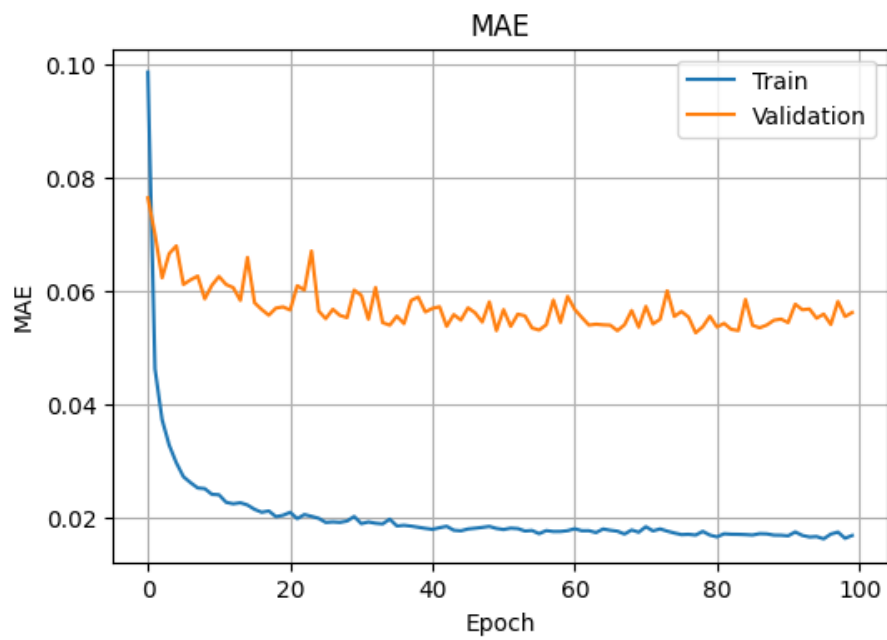
همچنین در شبکه‌های GRU تک‌جهته، آخرین خروجی زمانی ($output[:, -1, :]$) از نظر ریاضی معادل آخرین Hidden State است، اما استفاده مستقیم از $h_n[-1]$ تمیزتر، خواناتر و رایج‌تر است. در نتیجه، انتخاب Hidden Feature نهایی در این معماری نه تنها صحیح، بلکه از نظر مفهومی و عملی بهترین گزینه برای پیش‌بینی سری زمانی است.

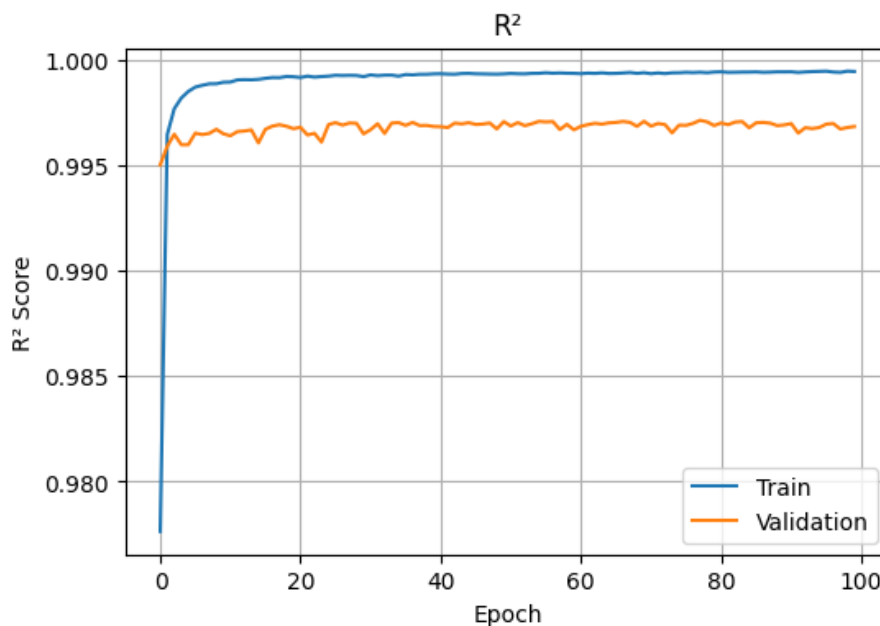
۲.۲.۳.۲

هایپرپارامترها

نمودارها







۴-۲ مقایسه روی داده‌های تست و مقایسه عملکرد

۱-۴-۲

جدول ۱۲: جدول مقایسه عملکرد مدل‌ها روی داده‌های تست

مدل	پارامترها	Test Loss (MSE)	R ² Score	MAPE (%)	MAE	RMSE
Baseline MLP	۲۰۵۴۸	۰.۰۲۵۳۴۹	۰.۹۹۱۷	۱.۲۷۹۶	۰.۱۱۵۹	۰.۱۵۷۲
CNN-LSTM	۶۹۰۶۰	۰.۰۰۸۹۵۹	۰.۹۹۶۹	۰.۷۲۶۶	۰.۰۶۴۴	۰.۰۹۶۱
CNN-GRU	۵۲۴۲۰	۰.۰۰۹۱۱۳	۰.۹۹۶۹	۰.۷۱۵۶	۰.۰۶۳۵	۰.۰۹۶۶

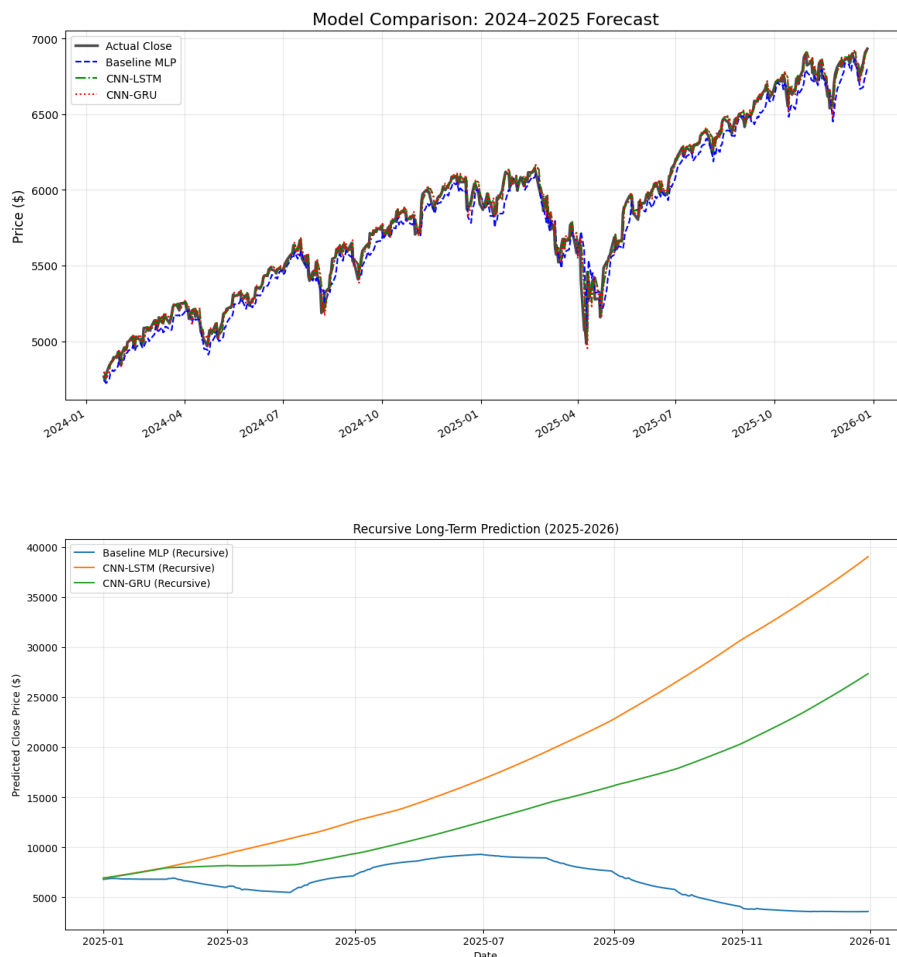
۲-۴-۲

جدول ۱۳: مقایسه کارایی محاسباتی مدل‌ها

مدل	پارامترها	تخمینی FLOP	آموزش زمان (ثانیه)
Baseline MLP	۲۰۵۴۸	۳۹۹۳۶	۱۹.۷۳
CNN-LSTM	۶۹۰۶۰	۱۲۱۸۸۱۶	۳۰.۶۱
CNN-GRU	۵۲۴۲۰	۹۲۳۹۰۴	۲۸.۸۶

سخت‌افزار مورد استفاده: GPU T4 × 2

۳-۴-۲



تحلیل و مقایسه عملکرد مدل‌ها (بازه ۲۰۲۴ تا ۲۰۲۵)

بر اساس نمودار پیش‌بینی و جدول متریک‌های ارزیابی، عملکرد مدل‌ها به صورت زیر تحلیل می‌شود:

مدل پایه MLP: (Baseline)

این مدل ضعیف‌ترین عملکرد را در میان مدل‌ها نشان داده است. همان‌طور که در نمودار مشاهده می‌شود، پیش‌بینی‌های این مدل در بسیاری از بازه‌ها، به‌ویژه در نوسانات شدید اواسط سال ۲۰۲۵، دچار کم‌برآوردی (Underestimation) شده و فاصله محسوسی با قیمت واقعی دارد. مقادیر بالاتر خطا و $MSE = 0.025$ (MAPE = 1.28%) نیز این ضعف را تأیید می‌کنند.

مدل‌های ترکیبی CNN-LSTM و CNN-GRU

هر دو مدل ترکیبی عملکردی بسیار دقیق و نزدیک به یکدیگر داشته‌اند. خروجی این مدل‌ها تقریباً منطبق بر قیمت واقعی است و توانسته‌اند هم روندهای کلی بازار و هم نوسانات کوتاه‌مدت را به خوبی مدل‌سازی کنند. در مقایسه جزئی‌تر، مدل CNN-LSTM کمترین مقدار MSE (0.0089) را ثبت کرده است، در حالی که مدل CNN-GRU با اختلافی بسیار ناچیز در MSE (0.0091)، در معیارهای MAE و MAPE عملکرد بهتری از خود نشان داده است.

انتخاب مدل برتر:

مدل CNN-GRU به عنوان مدل نهایی انتخاب می‌شود. دلیل این انتخاب، دستیابی به دقت بالا با پیچیدگی کمتر است. این مدل با تنها 52,420 پارامتر، عملکردی هم‌سطح و حتی بهتر (در معیار MAPE برابر با ۷۱.۰) نسبت به مدل CNN-LSTM با 69,060 پارامتر ارائه داده است که نشان‌دهنده کارایی بالاتر معماری GRU در این مسئله است.

محدودیت پیش‌بینی بلندمدت مستقیم

مدل‌های توسعه‌یافته در این پژوهش از نوع پیش‌بینی تک‌گام جلو (One-Step Ahead) هستند. این مدل‌ها برای پیش‌بینی قیمت روز آینده، به داده‌های واقعی چند روز گذشته (پنجره ورودی) وابسته‌اند. در نتیجه، امکان تولید پیش‌بینی مستقیم برای افق‌های زمانی بلندمدت (مانند یک سال آینده) وجود ندارد، زیرا داده‌های واقعی مورد نیاز برای ورودی مدل در دسترس نیستند.

راهکار: پیش‌بینی بازگشتی (Recursive Forecasting)

برای پیش‌بینی بازه‌های زمانی بلندتر (مانند ۲۰۲۶ تا ۲۰۲۷)، از روش پیش‌بینی بازگشتی استفاده می‌شود. در این روش، خروجی مدل در هر گام به عنوان ورودی گام بعدی در نظر گرفته می‌شود. به این ترتیب، پنجره ورودی به صورت تدریجی به جلو حرکت کرده و پیش‌بینی‌ها به صورت زنجیره‌ای تولید می‌شوند.

ملاحظه تحلیلی

ضعف مدل در پیش‌بینی‌های بلندمدت عمدتاً ناشی از ماهیت پویا و غیر ایستای بازارهای مالی و همچنین ساختار پیش‌بینی تک‌گام مدل است. این مدل‌ها بر اساس الگوهای کوتاه‌مدت گذشته آموزش می‌بینند و فرض می‌کنند که توزیع آماری داده‌ها در آینده مشابه گذشته باقی می‌ماند. در پیش‌بینی‌های بلندمدت، خروجی هر گام به عنوان ورودی گام بعدی استفاده می‌شود که باعث انباشت خطا (Error Accumulation) در طول زمان می‌گردد. علاوه بر این، تغییر رژیم‌های بازار (Market Regime Shifts)، وقوع شوک‌های ناگهانی و تغییر رفتار سرمایه‌گذاران

باعث می‌شود الگوهای تاریخی اعتبار خود را از دست بدهند. در نتیجه، مدل به تدریج دقت خود را از دست داده و پیش‌بینی‌ها به سمت میانگین یا روندهای ساده همگرا می‌شوند.

۲-۵ بخش امتیازی

۲-۵-۱

در این بخش، به جای استفاده از لایه‌های بازگشتی (RNN/LSTM/GRU) یا لایه‌های تمام‌متصل (Dense)، یک معماری کاملاً کانولوشنی (Fully Convolutional Network) طراحی شده است. در این معماری حتی لایه خروجی نیز به جای لایه خطی، با یک کانولوشن (1×1) پیاده‌سازی شده است که نقش نگاشت نهایی ویژگی‌ها به خروجی را بر عهده دارد.

تغییر ابعاد زمانی با Kernel Size و Stride

در شبکه‌های کانولوشنی، استخراج ویژگی و کاهش بعد زمانی به صورت همزمان انجام می‌شود. در معماری پیشنهادی، این هدف از طریق استفاده از Stride در لایه‌های کانولوشن محقق شده است.

استفاده از Stride: به جای به کارگیری Max Pooling که ممکن است بخشی از اطلاعات را حذف کند، در این شبکه از $\text{stride}=2$ در لایه‌های کانولوشن استفاده شده است. این کار باعث می‌شود فیلتر کانولوشن با گام‌های بزرگتر حرکت کند و طول دنباله زمانی در هر لایه تقریباً به نصف کاهش یابد (Downsampling کنترل‌شده). رابطه تغییر طول دنباله به صورت زیر است:

$$L_{\text{out}} = \left\lfloor \frac{L_{\text{in}} + 2 \times \text{padding} - \text{kernel_size}}{\text{stride}} + 1 \right\rfloor$$

اندازه هسته (Kernel Size): استفاده از کرنل با اندازه ۴ باعث می‌شود هر نورون در لایه بعدی، اطلاعات مربوط به ۴ گام زمانی متوالی را مشاهده کند. این موضوع موجب افزایش میدان دید زمانی (Receptive Field) و استخراج الگوهای کوتاه مدت معنادار می‌شود. به کارگیری $\text{padding}=1$ نیز کمک می‌کند اطلاعات مرزی دنباله حفظ شود.

فشرده‌سازی نهایی و تولید خروجی: پس از کاهش تدریجی طول توالی، از $\text{AdaptiveAvgPool1d}(1)$ برای فشرده‌سازی کامل بعد زمانی استفاده شده است. در نهایت، یک لایه Conv1d با کرنل (1×1) تعداد کانال‌ها را به تعداد خروجی مورد نظر (۴ متغیر) تبدیل می‌کند. این لایه عملاً نقش لایه Fully Connected را بدون از دست دادن خاصیت کانولوشنی شبکه ایفا می‌کند.

ساختار لایه‌ها و هایپرپارامترها

جدول ۱۴: ساختار لایه‌های شبکه CNN-Only

لایه نام	عملیات نوع	ورودی کانال	خروجی کانال	Kernel Size	Stride
Layer ۱	Conv ^۱ d	Features	۳۲	۴	۲
Act ۱	BatchNorm + ReLU	۳۲	۳۲	—	—
Layer ۲	Conv ^۱ d	۳۲	۶۴	۴	۲
Act ۲	BatchNorm + ReLU	۶۴	۶۴	—	—
Pooling	AdaptiveAvgPool ^۱ d	۶۴	۶۴	—	—
Layer ۳	Conv ^۱ d (1×1)	۶۴	۴	۱	۱

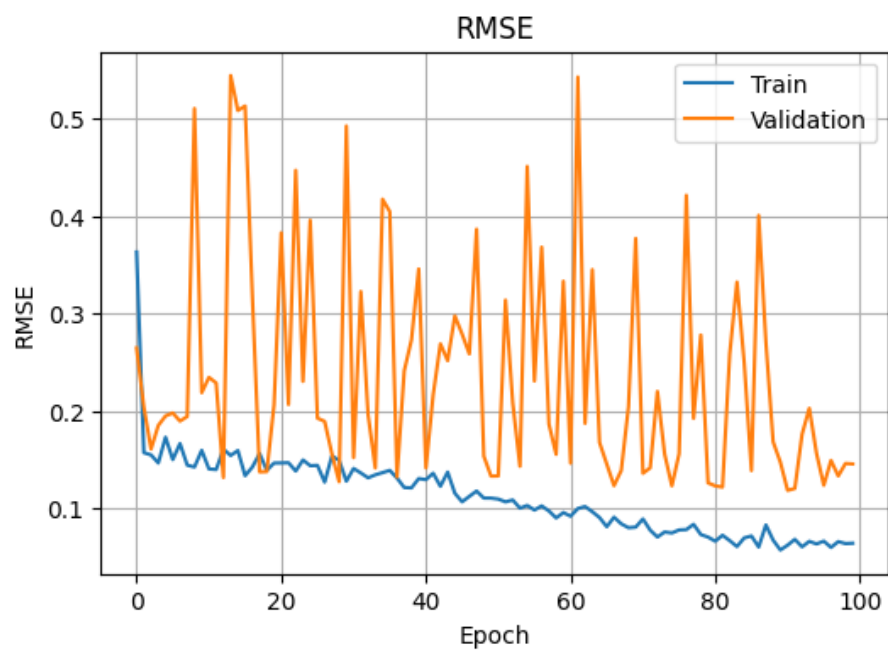
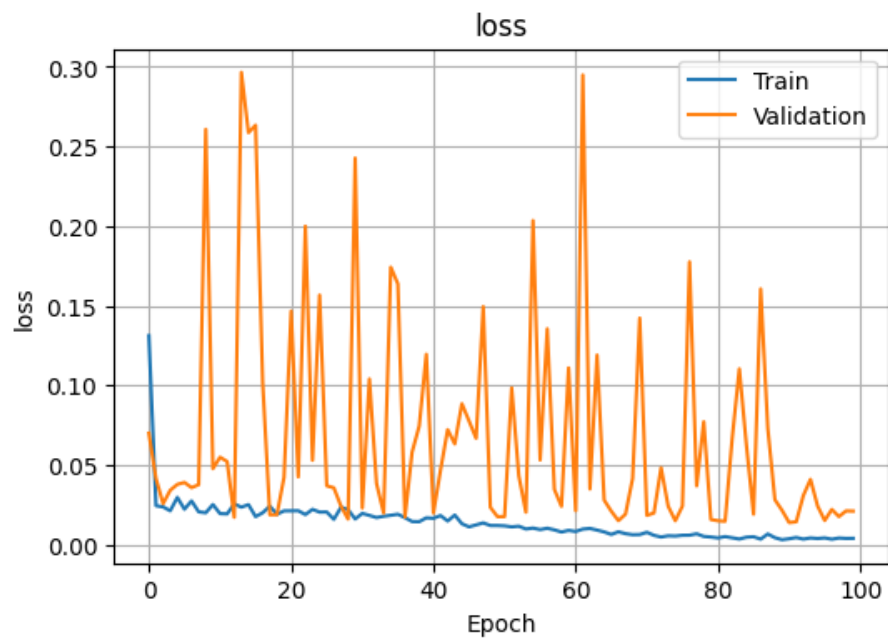
مقایسه حجم مدل با مدل پایه (Baseline)

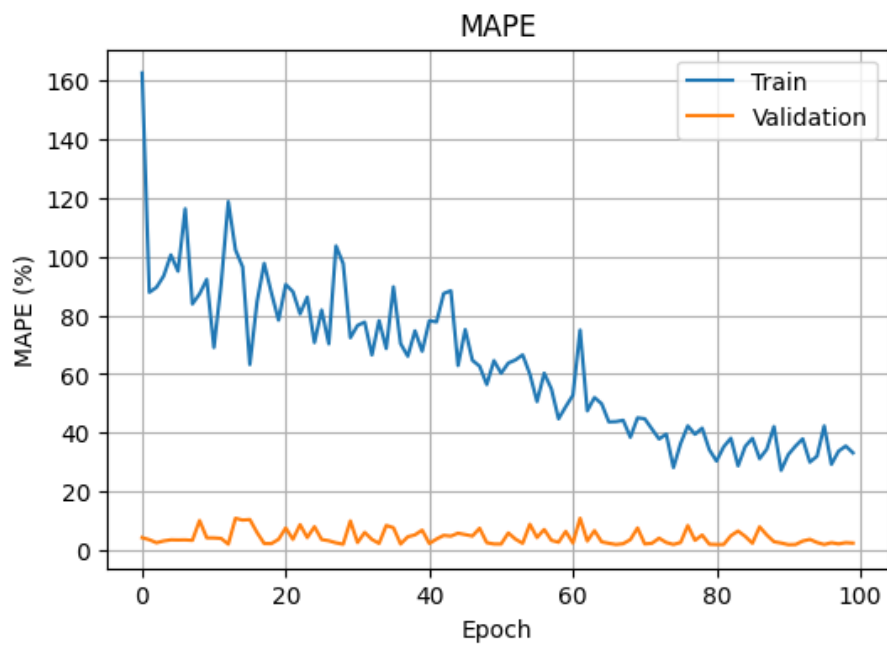
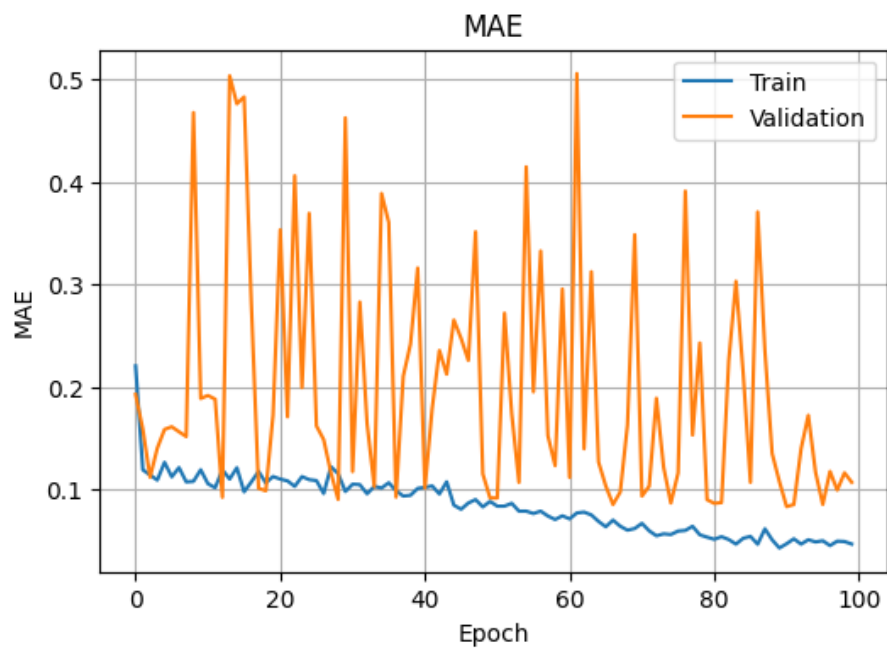
این معماری از نظر تعداد پارامترها به‌طور قابل توجهی سبک‌تر از مدل پایه است.

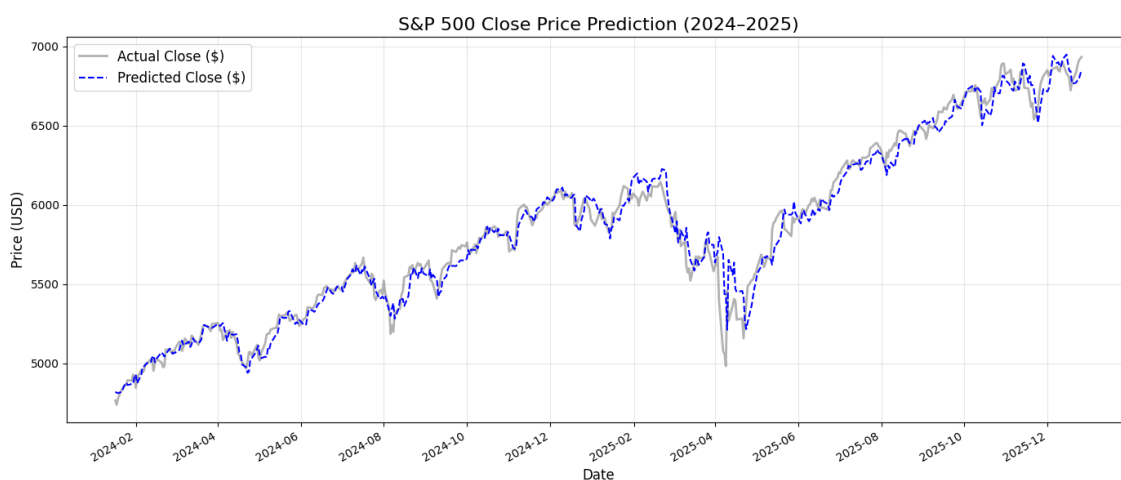
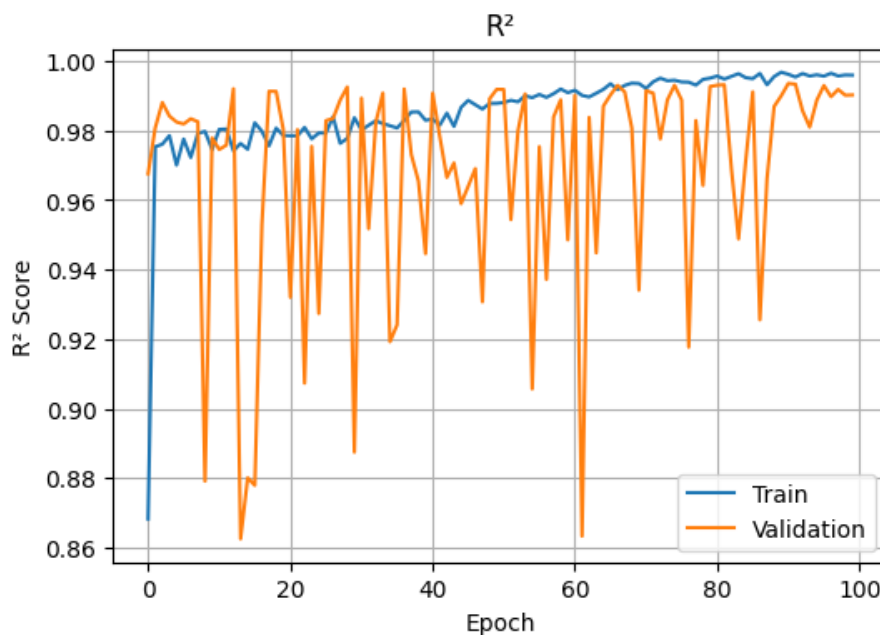
• **تعداد پارامترهای مدل CNN-Only:** حدود ۱۰۰۲۰ پارامتر

• **تعداد پارامترهای مدل Baseline MLP:** حدود ۲۰۵۴۸ پارامتر

تحلیل: مدل CNN-Only تقریباً ۵۰٪ پارامتر کمتر نسبت به مدل Baseline دارد. دلیل اصلی این کاهش، خاصیت اشتراک وزن‌ها (Weight Sharing) در لایه‌های کانولوشنی است. در حالی که در شبکه‌های MLP هر ورودی وزن مستقل دارد، در CNN یک فیلتر کوچک در کل توالی زمانی حرکت کرده و همان وزن‌ها بارها استفاده می‌شوند. این ویژگی علاوه بر کاهش تعداد پارامترها، باعث کاهش خطر بیش‌برازش (Overfitting) و افزایش کارایی محاسباتی مدل می‌شود.







ناپایداری و نوسان در فرآیند آموزش (Volatility)

برخلاف مدل‌های بازگشتی مانند LSTM و GRU که معمولاً روند همگرایی نرم و یکنواختی دارند، مدل CNN-Only در طول آموزش دچار نوسانات قابل‌توجهی شده است. به‌عنوان مثال، در حدود اپیاک ۳۰ مقدار خطای اعتبارسنجی (Validation Loss) به‌صورت ناگهانی تا حدود 0.24 افزایش یافته و هم‌زمان مقدار R^2 به حدود 0.88 کاهش پیدا کرده است. الگوی نوسان مشابهی در اپیاک‌های ۶۰ و ۷۰ نیز مشاهده می‌شود. علت اصلی این رفتار آن است که شبکه‌های کانولوشنی فاقد «حافظه ترتیبی» (Sequential State) هستند.

در نتیجه، این مدل‌ها نسبت به تغییرات ترکیب داده‌ها در هر Batch حساس‌تر بوده و ممکن است در فضای گرادین گام‌های بزرگ‌تری بردارند. این موضوع می‌تواند باعث عبور ناگهانی از نقاط بهینه (Local Minima) و ایجاد پرش در نمودار خطا شود.

آیا استفاده از معماری کاملاً کانولوشنی مناسب است؟

پاسخ نهایی: بله، استفاده از شبکه CNN-Only کاملاً مناسب است و در برخی شرایط حتی گزینه‌ای ترجیحی محسوب می‌شود.

اگرچه به‌طور سنتی تصور می‌شود که مدل‌سازی سری‌های زمانی نیازمند استفاده از ساختارهای بازگشتی مانند LSTM یا GRU است، نتایج این پروژه نشان می‌دهد که معماری CNN-Only نیز می‌تواند عملکردی قابل قبول و رقابتی ارائه دهد.

دلایل مناسب بودن CNN-Only:

* دقت بالا: دستیابی به مقدار $R^2 = 0.993$ نشان می‌دهد که مدل توانسته ساختار و الگوهای قیمتی را به‌خوبی یاد بگیرد. این عملکرد از مدل MLP Baseline بهتر بوده و به مدل‌های سنگین‌تر LSTM و GRU بسیار نزدیک است.

* کارایی و سبکی (Efficiency): این مدل تنها حدود ۱۰۰۰۰ پارامتر دارد که تقریباً یک‌پنجم تعداد پارامترهای مدل GRU است. در نتیجه، زمان آموزش و زمان استنتاج (Inference) به‌طور محسوسی کاهش یافته است. این ویژگی برای کاربردهایی مانند سیستم‌های معاملاتی سریع و بلادرنگ اهمیت بالایی دارد.

* تشخیص الگوهای محلی: شبکه‌های کانولوشنی در شناسایی الگوهای کوتاه‌مدت و محلی (مانند الگوهای چندروزه یا هفتگی قیمت) بسیار توانمند هستند. از آنجا که بازارهای مالی اغلب تحت تأثیر شوک‌ها و اخبار کوتاه‌مدت قرار می‌گیرند، این قابلیت یک مزیت مهم محسوب می‌شود.

نقطه ضعف و راهکار پیشنهادی:

تنها نقطه ضعف اصلی این معماری، ناپایداری در فرآیند آموزش و بروز پرش در نمودار Loss است. برای کنترل این مشکل در کاربردهای عملی، استفاده از Learning Rate Scheduler (کاهش تدریجی نرخ یادگیری در طول آموزش) توصیه می‌شود تا همگرایی مدل در ایپاک‌های پایانی پایدارتر شود.

جدول ۱۱: مشخصات معماری، پنجره‌های زمانی و ویژگی‌های ورودی مدل CNN-GRU

پارامتر	مقدار	توضیح
الگوریتم بهینه‌سازی	Adam	استفاده از <code>optim.Adam(model.parameters(), ...)</code>
نرخ یادگیری	۰.۰۱	مقدار پیش‌فرض در فرآیند آموزش
تابع خطا	MSE	میانگین مربعات خطا (<code>nn.MSELoss</code>)
تعداد Epoch	۱۰۰	تعداد تکرارهای آموزش مدل
اندازه Batch	متغیر	وابسته به <code>DataLoader</code> (معمولاً ۳۲ یا ۶۴)
کانال‌های ورودی	۱۰	برابر با تعداد ویژگی‌های ورودی
کانال‌های خروجی	۶۴	تعریف‌شده در <code>nn.Conv1d</code>
اندازه کرنل	۳	استخراج الگوهای کوتاه‌مدت زمانی
پدینگ	۱	حفظ طول دنباله زمانی
تابع فعال‌سازی	ReLU	اعمال پس از لایه کانولوشن
بعد فضای پنهان	۶۴	<code>64 = hidden_dim</code>
تعداد لایه‌ها	۲	پشته‌ای GRU (<code>num_layers = 2</code>)
درآپاوت	۲.۰	کاهش بیش‌برازش بین لایه‌ها
دوطرفه بودن	False	شبکه تک‌جهته
اندازه پنجره ورودی	۹	استفاده از ۹ روز گذشته به عنوان ورودی
اندازه پنجره خروجی	۱	پیش‌بینی یک گام جلوتر
نوع پیش‌بینی	Many-to-One	نگاشت چند روز ورودی به یک خروجی
متغیرهای قیمتی	Close Low, High, Open,	اطلاعات پایه بازار
ویژگی‌های زمانی و مهندسی‌شده	Day_sin, Month_cos, Month_sin, Day_cos, price_volume_interaction, high_low_ratio	کدگذاری چرخه‌ای زمان و ویژگی‌های ترکیبی
ساختار	GRU + CNN	استخراج الگوهای محلی و وابستگی زمانی
نوع پیش‌بینی	(Delta) Residual	پیش‌بینی تغییرات و جمع با قیمت آخر
نرمال‌سازی	Normalization Batch	اعمال بعد از CNN و GRU
بعد خروجی	۴	پیش‌بینی Close و Low High, Open